
LMGC90_GUI

Documentation Officielle

Version 0.6.0

© 2026 LMG90_GUI

Interface graphique · Matériaux · Modèles
Avatars · Boucles · Granulométrie
Conditions aux limites · Lois de contact
Maçonnerie · Corps déformables · Calcul
Post-traitement · Variables dynamiques · Architecture

TABLE DES MATIÈRES

- 1. Découverte de l'interface
- 2. Organisation d'un projet
- 3. Exemples
- 4. Matériaux
- 5. Variables dynamiques
- 6. Modèles
- 7. Avatars — Corps rigides
- 8. Avatar Vide
- 9. Boucles paramétriques
- 10. Granulométrie
- 11. Conditions aux limites (DOF)
- 12. Lois de contact
- 13. Assistant de maçonnerie
- 14. Corps déformables (EF)
- 15. Calcul (chipy)
- 16. Post-traitement
- 18. Visualisation
- 19. Architecture & Guide développeur

Chapitre 1

Découverte de l'interface

Découverte de l'interface graphique

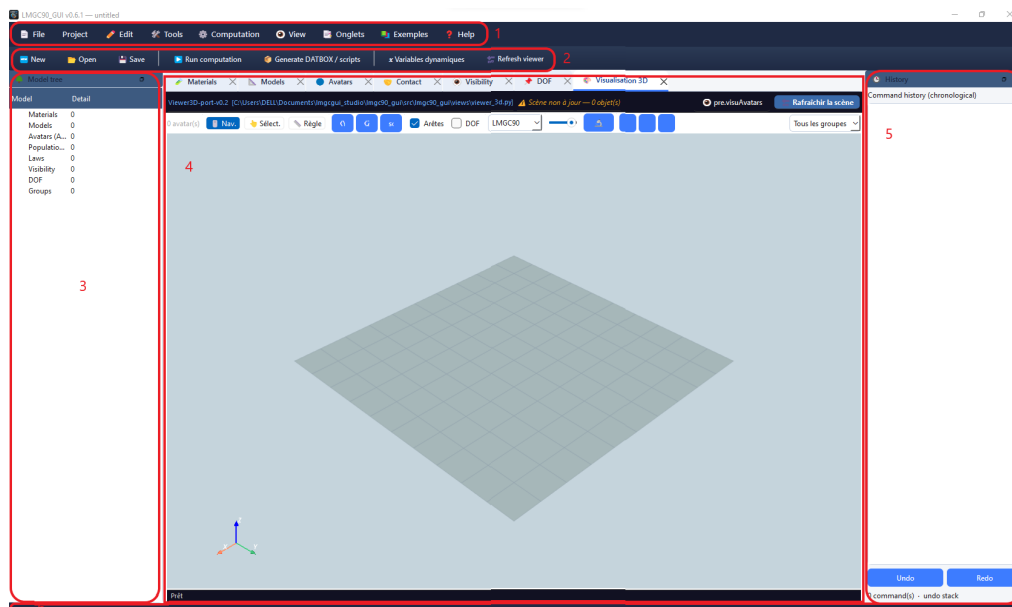
LMGC90_GUI est une interface graphique moderne conçue pour faciliter la création de modèles numériques avec le module **pre** (pré-processeur) de **LMGC90**.

Il est également possible de configurer et lancer vos calculs directement depuis LMG90_GUI via le module **chipy**.

L'interface est organisée de manière claire et ergonomique pour accompagner l'utilisateur du début à la fin du processus de modélisation : création des éléments, conditions aux limites, post-traitement, génération des fichiers et enfin lancement des calculs.

La vidéo suivante présente une vue d'ensemble des différentes parties de l'interface.

Fenêtre principale

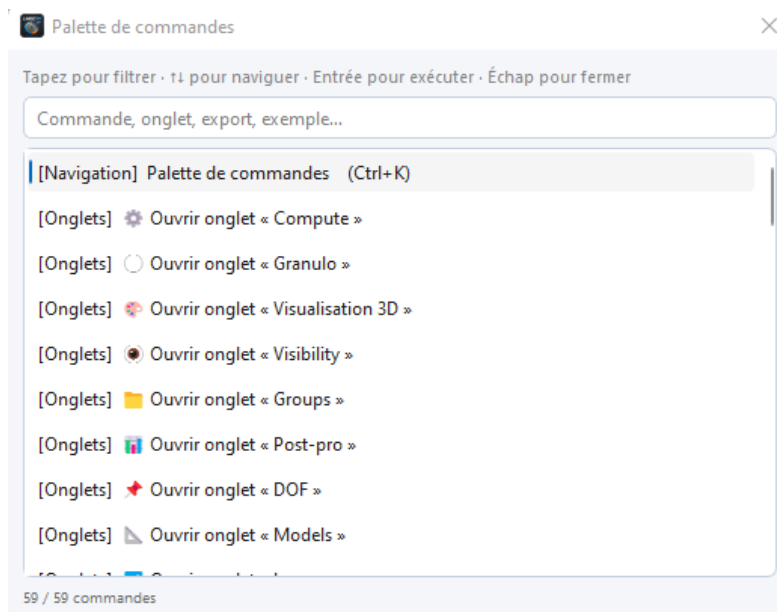


Vue globale de l'interface

L'interface est divisée en **Six zones principales** :

1. Menu
2. Barre d'outils (en haut)
3. Arbre du modèle (à gauche)
4. Onglets de création (centre)
5. Historique des commandes (à droite)
6. Barre d'état (en bas)

La palette des commandes, est accessible dans la barre d'outils ou avec le raccourci clavier **ctrl+K**.



Palette des commandes

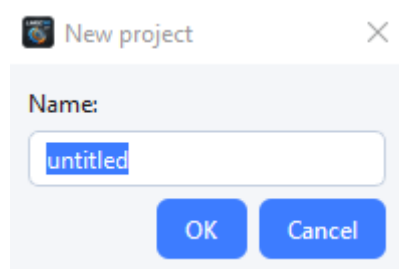
1. Menu

1.1 Fichier

Action	Raccourci	Description
Nouveau	Ctrl+N	Crée un nouveau projet. Une boîte de dialogue s'ouvre pour saisir la dimension et le nom du projet.
Ouvrir	Ctrl+O	Ouvre un projet existant. Une boîte de dialogue permet de naviguer jusqu'au fichier <code>.lmgc90</code> du projet.
Sauvegarder	Ctrl+S	Sauvegarde le projet courant à son emplacement actuel.
Sauvegarder sous...	Ctrl+Shift+S	Sauvegarde le projet sous un nouveau nom ou dans un nouvel emplacement.
Quitter	Ctrl+Q	Ferme l'application.

Nouveau projet

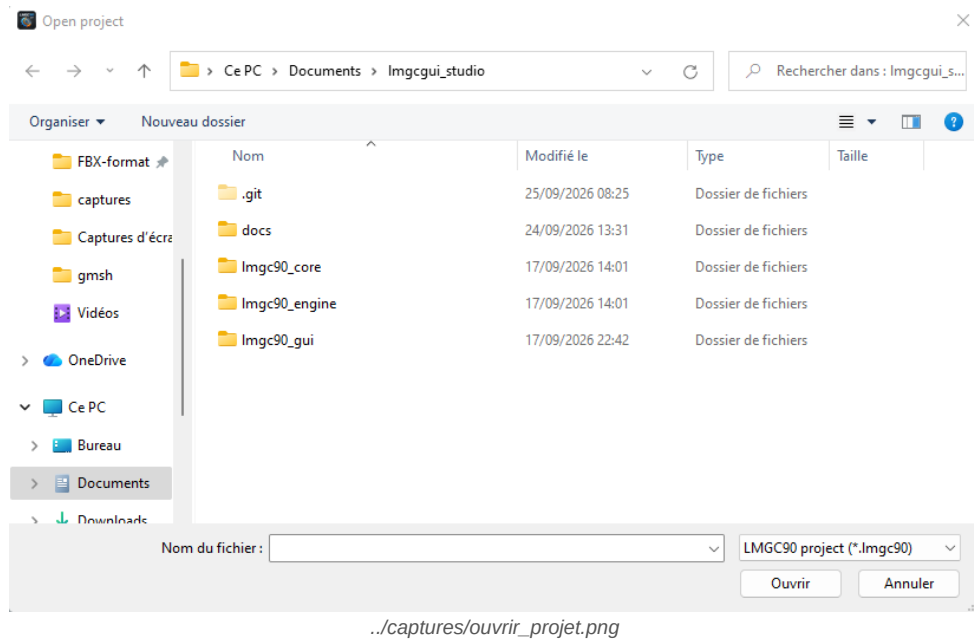
Cliquez sur le bouton **Nouveau** de la barre d'outils, ou utilisez le menu **Fichier** → **Nouveau**, ou appuyez sur **Ctrl+N**. Une boîte de dialogue s'ouvre pour renseigner la dimension puis nom du projet



../captures/titre_projet.png

Ouvrir un projet

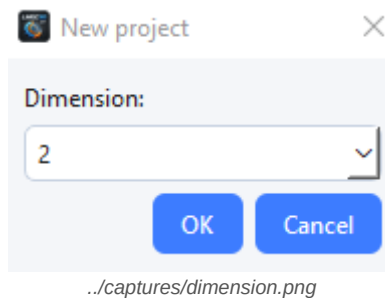
Cliquez sur le bouton **Ouvrir** de la barre d'outils, utilisez le menu **Fichier** → **Ouvrir**, ou appuyez sur **Ctrl+O**. Il vous suffira ensuite de spécifier le chemin et le nom de votre projet



Sauvegarder Sert à sauvegarder vos projets dans votre disque dur, cliquez sur le bouton **sauvegarder** de la barre d'outils, ou de cliquez sur le menu **Fichier-> Sauvegarder**, ou avec le raccourci clavier **Ctrl+S**,

1.2 Projet

Définir la dimension Afin de rendre la dimension la même dans tout votre projet vous pouvez cliquer sur **Projet-> définir la dimension**



1.3 Edition

Action	Raccourci	Description
Annuler	Ctrl+Z	Annuler une action.
Rétablir	Ctrl+Y	Rétablir l'action anulée.

1.4 Outils

Action	Raccourci	Description
Générer pre.py	—	Génère votre script modèle (pre.py).

Action	Raccourci	Description
générer command.py	—	Génère votre script calcul (command.py).
Exporter tout	—	Exporte scripts modèle/calcul et DATBOX
Pipeline	—	Génère votre script sbatch pour une machine HPC
Assistant déformable	Ctrl+Shift+D	Guide la création ou l'importation d'éléments déformables (maillages rectangles, disques, sphères, cylindres ou fichiers externes .msh / .geo, etc.).
Assistant maçonnerie	Ctrl+Shift+M	Spécialisé dans la création d'empilements de briques 2D et 3D (brick2D / brick3D) selon différents appareillages (standard, running bond, paneressse simple, paneressse double, etc.).
Variables dynamiques	Ctrl+V	Ouvre une boîte de dialogue permettant de définir des variables réutilisables dans les champs numériques de l'interface (rayon, espacement, offset, etc.). C'est également une fenêtre d'inspection des propriétés des objets LMGC90 présents en mémoire. Voir la page Variables dynamiques.
visuAvatars (pylmgc90)	—	Permet de visualiser votre modèle avec la fonction native de LMGC90 <code>`visuAvatars`</code>
Préférences	Ctrl+,	Ouvre la boîte de dialogue de configuration de l'application. Voir section Préférences.

Remarque : les assistants peuvent être relancés à tout moment pendant la session. Chaque exécution ajoute les éléments générés à la suite du projet existant, sans effacer ce qui a été créé auparavant.

La duplication des éléments qui portent le même nom entraîne des erreurs, pensez à nommer différemment vos éléments.

1.5 Calcul

Action	Raccourci	Description
Générer DATBOX et scripts	Ctrl+F5	Génère le DATBOX et les scripts modèle/calcul
Lancer calcul	F5	Ouvre l'onglet calcul et lance le calcul chipy directement depuis l'interface, dans un processus séparé pour ne pas bloquer l'interface.
Journal de l'application	F7	Affiche le journal interne de LMGC90_GUI : erreurs non gérées, avertissements Python, appels pylmgc90 échoués. Utile pour diagnostiquer les problèmes qui ne génèrent pas de message visible dans l'interface.



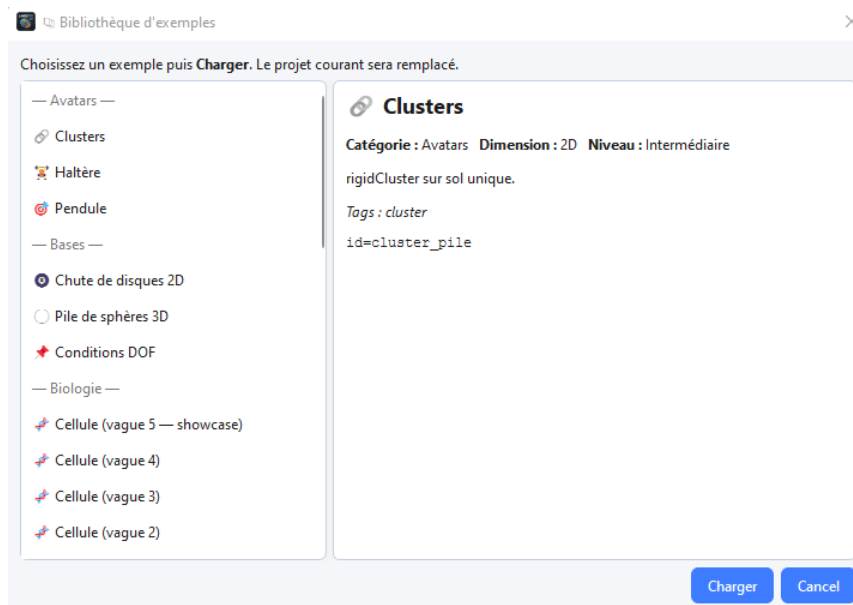
../captures/journal.png

1.6 Onglets

Action	Raccourci	Description
Ouvrir	—	Ouvre un onglet spécifique parmi la liste complète. Voir section Onglets de création.
Fermer les autres	—	Ferme tous les onglets ouverts sauf l'onglet actif.
Fermer tous (sauf essentiels)	—	Ferme tous les onglets non essentiels.
Onglets par défaut	Ctrl+Alt+D	Restaure la disposition d'onglets par défaut.

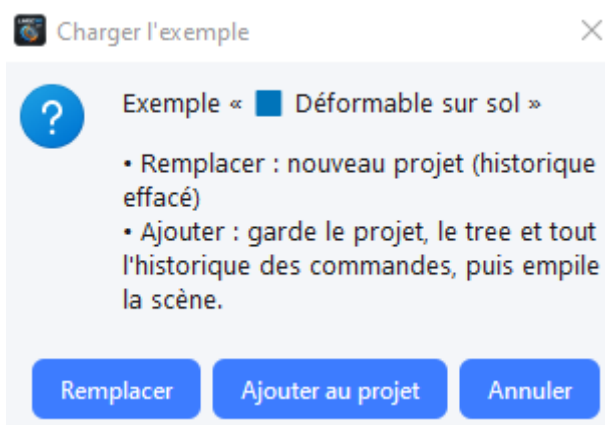
1.7 Exemples

Ce menu contient une seule action qui est **Bibliothèque d'exemples**, elle ouvre une boîte de dialogue dans laquelle vous pouvez parcourir plusieurs exemples sous sept catégories. La sélection d'un exemple charge les détails de cet exemple sur l'onglet de droite.



../captures/biblio_exemples.png

Afin de charger un exemple, il vous suffit simplement donc de le sélectionner et cliquer sur le bouton **Charger**, dans mon cas je vais charger l'exemple **Corps déformable sur sol rigide**, un nouvelle boîte de dialogue s'ouvrira afin de **remplacer**, **d'ajouter au projet** qui conduira à des éléments en double ou **d'annuler** l'action.



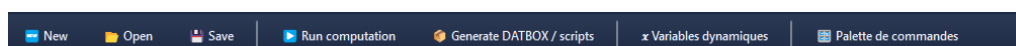
../captures/exemple_capture.png

1.8 Aide

Action	Description
Palette des commandes	Ouvre la palette des commandes.
À propos	Affiche les informations sur la version de LMGC90_GUI et les dépendances.
Aide en ligne	Ouvre la documentation en ligne dans le navigateur par défaut.

2. Barre d'outils

La barre d'outils regroupe les actions les plus fréquentes pour un accès rapide :

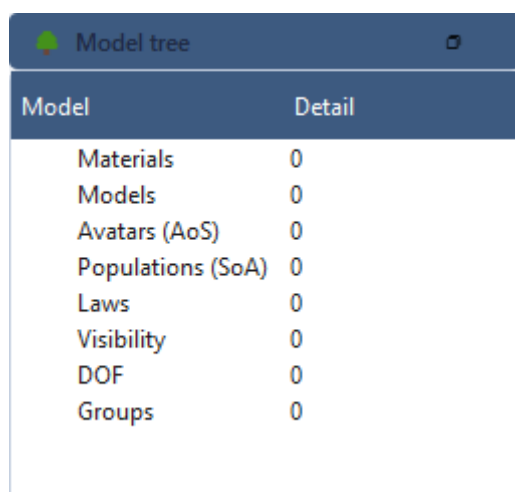


../captures/barre_outils.png

Bouton	Équivalent menu
Nouveau	Fichier → Nouveau
Ouvrir	Fichier → Ouvrir
Lancer le calcul	calcul → Lancer le calcul
Sauvegarder	Fichier → Sauvegarder
Générer DATBOX/scripts	Calcul → Générer DATBOX/scripts
Variables dynamiques	Outils → Variables dynamiques
Palette des commandes	Aide → Palette des commandes

3. Arbre du modèle (à gauche)

Zone fixe affichant l'**arborescence du modèle**. Elle se met à jour automatiquement après chaque création, modification ou suppression d'élément.



Model tree	
Model	Detail
Materials	0
Models	0
Avatars (AoS)	0
Populations (SoA)	0
Laws	0
Visibility	0
DOF	0
Groups	0

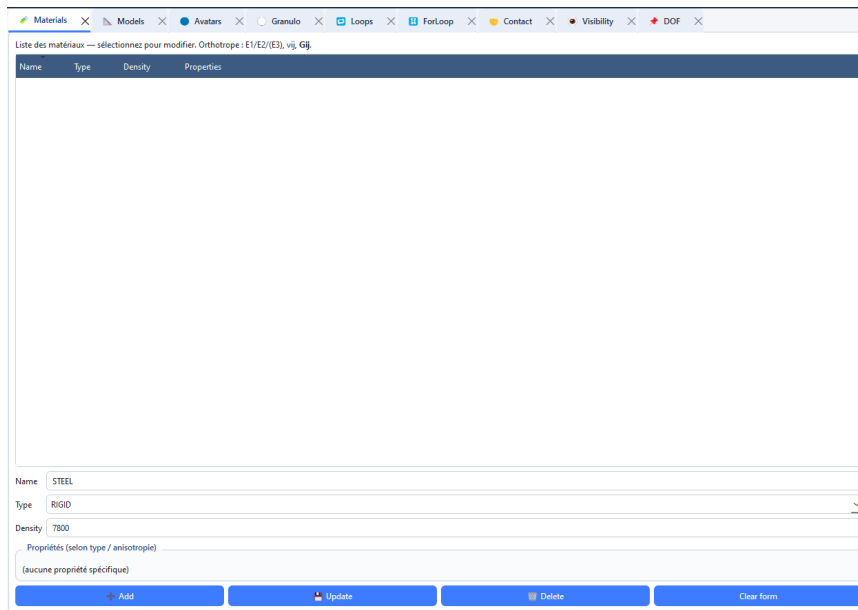
../captures/model_tree.png

Sections affichées

Section	Contenu
Matériaux	Liste de tous les matériaux définis dans le projet.
Modèles	Liste de tous les modèles éléments finis (physique, élément, dimension).
Avatars (AOS)	Liste de tous les corps du projet : rigides, vides, déformables, Boucle, ForLoop, etc.
Population d'avatars (SOA)	Liste la granulométrie ou les assistants .
Lois de contact	Lois de comportement de contact définies (frottement, cohésion, rigidité).
Tables de visibilité	Règles de visibilité entre avatars pendant le calcul.
DOF	Liste les CL sur les avatars ou groupes d'avatars.
Groupes	Liste les groupes d'avatars.

4. Onglets de création (zone centrale)

Zone principale de travail. Chaque onglet est dédié à une étape de modélisation. Pour ouvrir un onglet, utilisez le menu **Onglets** → **Ouvrir** et choisissez l'onglet souhaité.



../captures/onglets.png

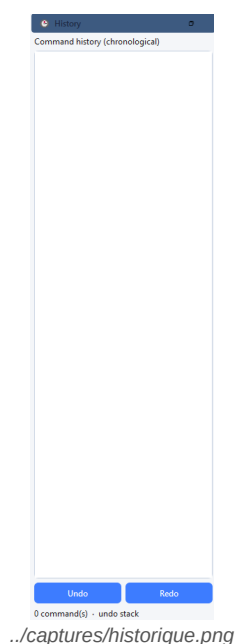
Onglet	Raccourci	Description
Matériau	Ctrl+1	Création et gestion des matériaux (RIGID, ELAS, ELAS_PLAS, THERMO_ELAS, PORO_ELAS, etc.).
Modèle	Ctrl+2	Définition des modèles physiques et éléments finis (MECAx, THERx, POROx, MULTI).
Avatar	Ctrl+3	Création de corps rigides standards : disque, jonc, polygone, mur rugueux, sphère, cylindre, polyèdre, etc.
Boucles	Ctrl+4	Génération paramétrique de séries d'avatars : cercle, grille, ligne, spirale ou placement manuel.
ForLoop	Ctrl+5	Crée des boucles For sur vos éléments : matériaux, modèles, avatars, DOF, etc.
Granulométrie	Ctrl+6	Génération de dépôts avec distribution statistique des rayons et dépôt gravitaire.
Contact	Ctrl+7	Définition des lois de comportement de contact (frottement Coulomb, cohésion, rigidité normale et tangentielle).
Visibilité	Ctrl+8	Création de tables de visibilité avec lesquels interagissent les avatars pendant le calcul.
DOF	Ctrl+9	Conditions aux limites : translations imposées, rotations bloquées, vitesses imposées, couplages de degrés de liberté.
Postpro	—	Configuration des commandes de post-traitement : bilan énergétique, suivi de corps, extraction de champs.

Onglet	Raccourci	Description
Visualisation 3D	—	Affichage interactif des avatars du modèle avec modes de navigation, sélection et mesure.
Groupes	—	Paramétrer vos groupes d'avatars.
Contacteurs	—	Permet d'ajouter/supprimer des contacteurs pour vos avatars.
Assistant déformables	—	Assisant pour la création/importation des éléments déformables.
Assistant maçonnerie	—	Assistant pour la création et configuration d'éléments de maçonnerie.

Raccourcis clavier : les touches **Ctrl+1** à **Ctrl+9** ouvrent directement les neuf premiers onglets de la liste.

5. Zone historique des commandes (à droite)

Zone dédiée à la gestion de l'historique des commandes.



6. Barre d'état (en bas)

Bande horizontale en bas de la fenêtre affichant des messages contextuels sur les opérations en cours : création d'un avatar, génération d'un script, résultat d'une mesure dans le viewer 3D, erreur de validation, etc.



Modes interactifs du viewer 3D

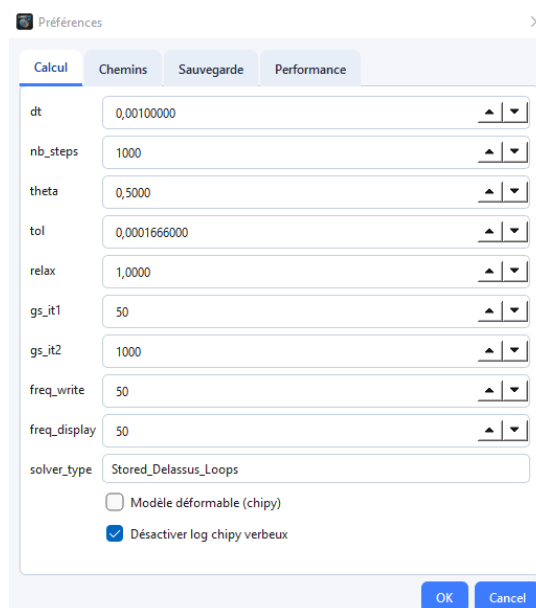
Mode	Description
■ ■ Navigation	Mode par défaut : rotation (clic gauche + glisser), zoom (molette), panoramique (clic droit + glisser).

Mode	Description
■ Sélection	Clic sur un avatar pour le mettre en évidence (surlignage jaune) et afficher ses informations dans la barre d'état.
■ Règle	Mesure de distance : cliquer sur un premier point (A) puis un second point (B) pour afficher la distance en mètres.

Les vues rapides **XY**, **XZ**, **YZ** et **Iso** sont accessibles depuis la barre d'outils du viewer.

Préférences

Accessible via **Outils** → **Préférences** ou le raccourci **Ctrl+,**. La boîte de dialogue de préférences regroupe les paramètres de configuration de l'application.



../captures/preferences.png

Paramètre	Description
Calcul	Contient les valeurs de calculs par défauts
Chemins	Chemin par défaut utilisé lors de l'ouverture et de la sauvegarde des projets.
Sauvegarde	Options pour activer la sauvegarde automatique à intervalles réguliers et à la fermeture de l'application.
Performances	Active ou désactive l'affichage des avatars dans l'arbre du modèle et dans le tableau de l'onglet Avatar.

Résumé des raccourcis clavier

Raccourci	Action
Ctrl+N	Nouveau projet
Ctrl+O	Ouvrir un projet
Ctrl+S	Sauvegarder
Ctrl+Shift+S	Sauvegarder sous...

Raccourci	Action
Ctrl+Q	Quitter
Ctrl+Shift+N	Assistant de configuration de projet
Ctrl+Shift+G	Assistant de granulométrie pylmgc90
Ctrl+Shift+D	Assistant de déformable
Ctrl+Shift+M	Assistant de maçonnerie
Ctrl+V	Variables dynamiques
Ctrl+,	Préférences
Ctrl+F5	Paramètres de calcul
F5	Lancer le calcul
F6	Voir les logs LMGC90
F7	Journal de l'application
Ctrl+Alt+D	Onglets par défaut
Ctrl+1 ... Ctrl+9	Ouvrir l'onglet correspondant
Ctrl+K	Ouvrir la palette des commandes

LMGC90_GUI est conçue pour être **intuitive** et **entièrement visuelle**, tout en conservant une compatibilité totale avec les scripts Python traditionnels de LMGC90.

Pour apprendre les opérations spécifiques, consultez les chapitres de l'index français.

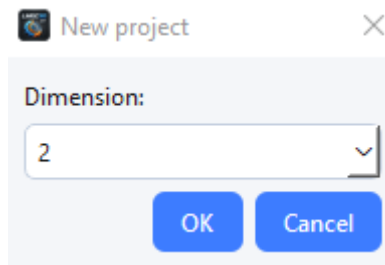
Organisation d'un projet

Créer, ouvrir et organiser un projet

Cette section parlera de tout ce qui est création/ouverture et sauvegarde d'un projet.

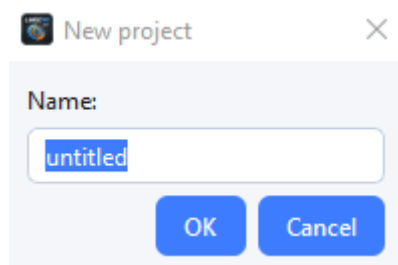
Créer un projet

1. Choisissez **File** → **New** ou appuyez sur **Ctrl+N**.
2. Dans la boîte **New project**, choisissez la dimension **2** ou **3**.



../captures/dimension.png

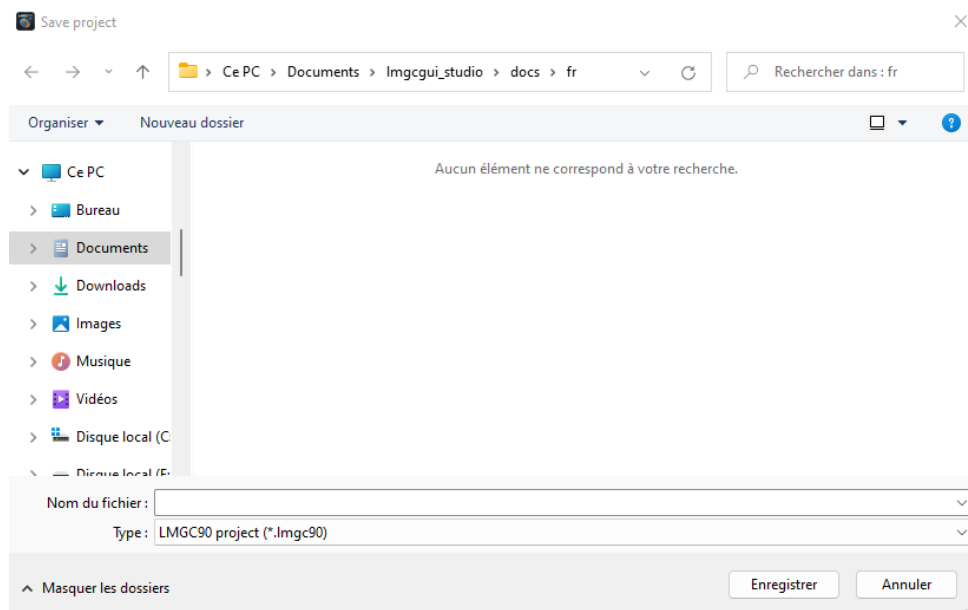
3. Saisissez un nom puis validez. Le projet est créé dans l'application ; il n'est pas encore enregistré sur disque.



../captures/titre_projet.png

Afin d'enregistrer votre projet :

4. Choisissez **File** → **Save As...** (**Ctrl+Shift+S**), sélectionnez un dossier et un nom de fichier, puis enregistrez. L'extension **.lmgc90** est ajoutée si elle manque.



../captures/save_projet.png

5. Vérifiez le nom et la dimension dans la barre d'état.

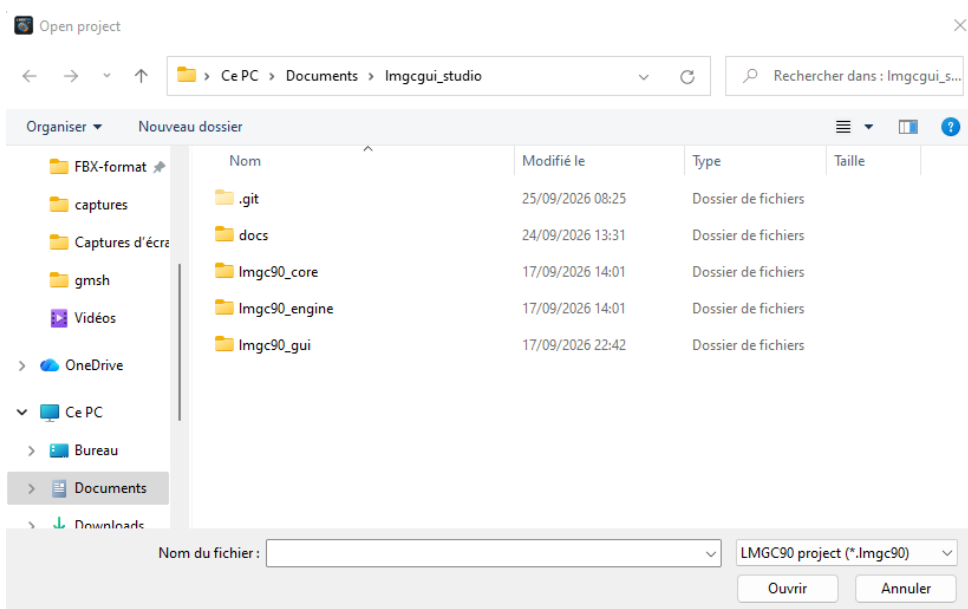
Enregistrer et reprendre le travail

Une fois votre projet enregistré sur votre disque dur avec ses deux commandes :

- **File** → **Save** (**Ctrl+S**) enregistre les modifications à l'emplacement courant.
- **File** → **Save As...** crée une copie sous un autre nom ou dans un autre dossier.

Il est tout à fait possible de l'ouvrir et de lui apporter des modifications:

- **File** → **Open...** (**Ctrl+O**) ouvre un fichier de projet **.lmgc90**.



../captures/ouvrir_projet.png

Vous pouvez bien sûr annuler ou rétablir des opérations sur vos modèles.

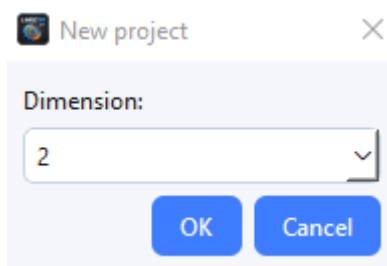
- **Edit** → **Undo** (**Ctrl+Z**) et **Edit** → **Redo** (**Ctrl+Y**) annulent ou rétablissent les dernières actions lorsque l'historique les permet.

Remarque: Enregistrez le projet après les opérations importantes, avant de lancer un calcul et avant de charger un exemple en mode remplacement.

Choisir et modifier la dimension

La dimension du projet détermine les éléments, avatars, contacteurs et corps proposés dans les formulaires.

1. Définissez **2** ou **3** au moment de la création.
2. Pour la modifier ensuite, choisissez **Project** → **Set dimension...**



../captures/dimension.png

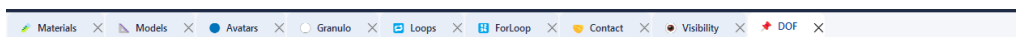
3. Sélectionnez la nouvelle dimension et lisez l'avertissement si des modèles existants ne correspondent plus.
4. Confirmez uniquement après avoir vérifié les modèles et les avatars. Le changement de dimension ne convertit pas automatiquement les objets existants.

En cas de changement majeur entre 2D et 3D, la solution la plus claire est souvent de créer un nouveau projet de la bonne dimension et d'y reconstruire ou recharger la scène appropriée.

Première séquence de modélisation

Une fois le projet créé, le parcours usuel dans les onglets est :

1. Créer un matériau dans **Materials**.
2. Créer un modèle dans **Models**.
3. Créer un ou plusieurs corps dans **Avatars**, ou utiliser **Granulo**, pour une génération granulométrique.
4. Créer des **Boucles paramétriques** et **Boucles Formelles** sur vos éléments.
5. Définir les lois de contact dans **Contact**, puis leurs associations dans **Visibility**.
6. Ajouter les conditions aux limites dans **DOF** et, si utile, créer des **Groups**.
7. Enregistrer le projet, configurer **Compute**, puis préparer et lancer le calcul.

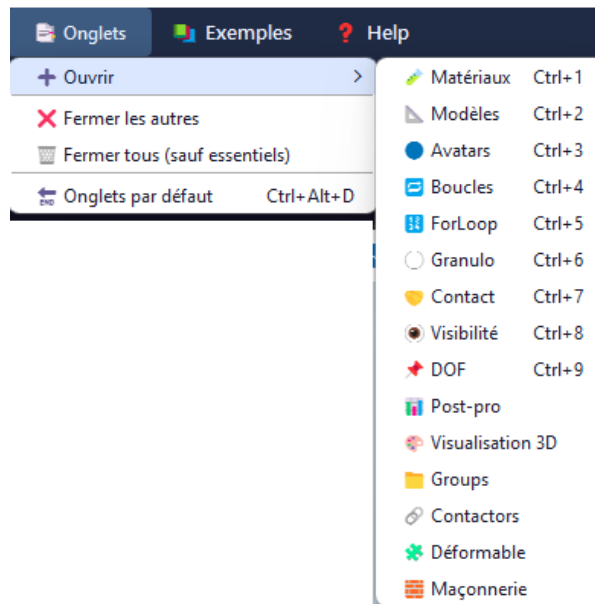


../captures/onglets_list.png

Les assistants **Assistant Mesh déformable** (**Ctrl+Shift+D**) et **Assistant Maçonnerie** (**Ctrl+Shift+M**) sont disponibles depuis **Tools**. L'assistant général de configuration de projet et l'assistant Granulométrie décrits dans les anciennes pages ne font pas partie des menus actuels.

Organisation des onglets

Utilisez **Onglets** → **Ouvrir** pour afficher une fonction. Les onglets **Materials** et **Models** restent ouverts comme onglets essentiels. Fermer un onglet ne supprime pas les éléments du projet.



../captures/onglets_open.png

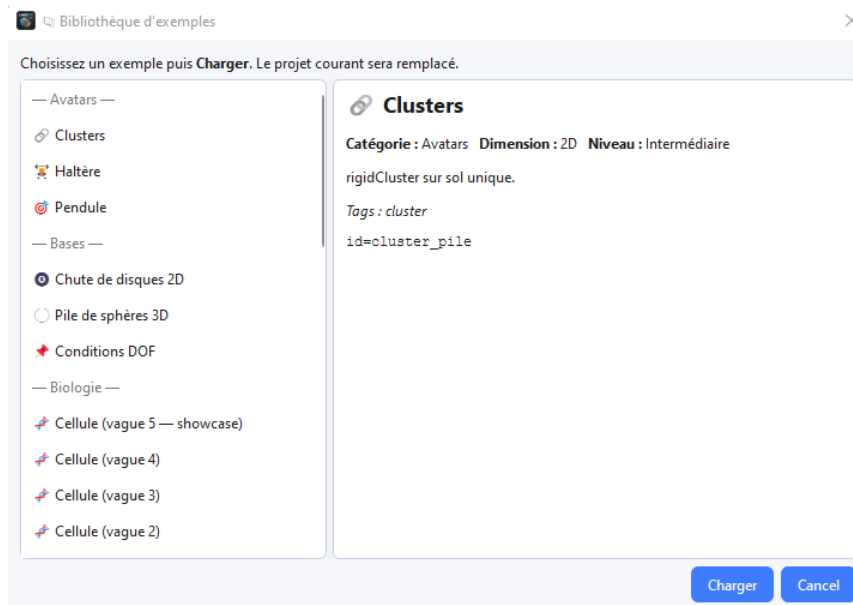
Les neuf premiers onglets ont les raccourcis **Ctrl+1** à **Ctrl+9** dans cet ordre : Materials, Models, Avatars, Loops, ForLoop, Granulo, Contact, Visibility et DOF. **Onglets** → **Onglets par défaut (Ctrl+Alt+D)** restaure la disposition initiale.

Pour le détail des menus, panneaux et raccourcis, voir Découverte de l'interface. Pour les procédures de création, utilisez les chapitres spécialisés listés dans l'index.

Bibliothèque d'exemples

Documentation de quelques exemples fournis avec l'application, accessibles via **■ Exemples → Parcourir les exemples...** (Ctrl+Shift+E).

La bibliothèque permet de parcourir des scènes de démonstration et de les charger dans un projet. Elle sert de point de départ graphique ; inspectez et enregistrez le projet après le chargement.



exemples fournis

Table des matières

1. Vue d'ensemble
2. Bases
 - Chute de disques 2D
 - Empilement de sphères 3D
3. Structures
 - Mur de maçonnerie
 - Mur avec liaisons cohésives (CZM)
4. Génération de masse
 - Dépôt granulométrique
 - Boucle géométrique — Cercle
5. Avancé — Contact et mécanismes
 - Tambour rotatif
 - Décharge en trémie
 - Contact roue/rail (ferroviaire)

- Frein à disque de vélo
 - Roulement à billes (coupe 2D)
6. Avancé — Déformable
 - Corps déformable sur sol rigide
 7. Synthèse
 - Scène composite
 8. Tableau récapitulatif complet

Parcourir le catalogue

1. Choisissez **Exemples** → **Bibliothèque d'exemples...** (**Ctrl+Shift+E**).
2. Parcourez la liste groupée par catégorie.
3. Cliquez sur un exemple. Le panneau de droite affiche son titre, sa dimension, son niveau, sa description et ses mots-clés.
4. Cliquez sur **Charger** pour poursuivre ou **Cancel** pour fermer sans sélectionner d'exemple.

Remarque : Avant de choisir **Remplacer**, enregistrez le projet courant si vous souhaitez le garder. Avant **Ajouter au projet**, pensez aux noms déjà présents : des collisions peuvent empêcher une fusion correcte.

Après le chargement

1. Vérifiez le nom du projet et sa dimension dans la barre d'état.
2. Parcourez le contenu dans **Model tree** et dans les onglets Materials, Models, Avatars, Contact et Visibility.
3. Cliquez sur **Rafraîchir la scène** dans **Visualisation 3D** pour afficher les avatars/populations de l'exemple.
4. Enregistrez le projet avec **File** → **Save As...** pour garder une copie de travail indépendante.

Les scènes disponibles et leurs catégories évoluent avec l'application.

2. Bases

2.1 Chute de disques 2D

Dimension : 2D · **Difficulté :** Débutant

Le point de départ classique : une rangée de disques rigides tombe sous gravité sur un mur.

Élément	Détail
Matériau	TDURx , RIGID, $\rho = 2500 \text{ kg/m}^3$
Modèle	rigid , MECaX / Rxx2D
Sol	SMOOTH_WALL (mur lisse fixe), $l=4.0$, $h=0.1$
Avatars mobiles	10 RIGID_DISK ($r = 0.1 \text{ m}$) via boucle Ligne (step=0.3)
Loi de contact	IQS_CLB , friction = 0.3
Visibilité	Disque/Disque + Disque/Sol (contacteurs DISKx/JONCx)

Mécanismes illustrés : **Loop** géométrique de type Ligne, création d'un avatar modèle puis génération automatique des copies via `controller.generate_loop(loop)`.

2.2 Empilement de sphères 3D

Dimension : 3D · **Difficulté** : Débutant

Grille 3×3 de sphères rigides empilées sur un plan.

Élément	Détail
Matériau	TDURx, RIGID, $\rho = 2500 \text{ kg/m}^3$
Modèle	rigid, MECaX / Rxx3D
Sol	RIGID_PLAN (axe1=axe2=2.0, axe3=0.05)
Avatars	9 RIGID_SPHERE ($r = 0.15 \text{ m}$), grille 3×3, pas = 0.6 m
Lois	IQS_CLB, friction = 0.3
Visibilité	Sphère/Sphère + Sphère/Plan (SPHER/PLANx)

Note d'implémentation : placement **direct** des sphères (boucle Python `for`), car le générateur `Loop` "Grille" intégré ne produit que des centres 2D — voir `core/generators.py::LoopGenerator.generate_grid`.

3. Structures

3.1 Mur de maçonnerie

Dimension : 2D · **Difficulté** : Intermédiaire

Mur de 8 rangs × 5 colonnes de briques en appareil **Standard** (décalage d'une demi-brique entre rangs consécutifs).

Élément	Détail
Matériau	brick, RIGID, $\rho = 1800 \text{ kg/m}^3$
Brique	pre.brick2D("std", lx=0.20, ly=0.065) (dimension française standard)
Représentation	EMPTY_AVATAR avec wall_params={'l', 'h', 'brick_name'}
Groupe	mur_briques (tous les avatars générés)
Loi de contact	IQS_CLB, friction = 0.6

Mécanisme clé : chaque brique est créée en deux temps — objet `pymgc90` réel via `pre.brick2D(...).rigidBrick(...)` (ajouté directement à `controller._bodies_container`), puis un `Avatar(EMPTY_AVATAR)` correspondant dans `state.avatars` pour la persistance/l'UI.

3.2 Mur avec liaisons cohésives (CZM)

Dimension : 2D · **Difficulté** : Avancé

Deux rangées de 6 blocs collées avec une loi de **zone cohésive** (résistance avant rupture, puis dégradation).

Élément	Détail
Brique	0.25 × 0.10 m, 2 assises × 6 colonnes
Loi de contact	IQS_MAC_CZM
Propriétés obligatoires	stfr=1e10, dyfr=1e10, cn=5e6, ct=3e6, b=1.0, w=0.02

Point d'attention documenté dans le code source : les propriétés `IQS_MAC_CZM` doivent être passées via `properties={...}` et **jamais** comme paramètres directs du constructeur `ContactLaw` (piège déjà rencontré et corrigé dans `ex_deformable_drop.py`).

4. Génération de masse

4.1 Dépôt granulométrique

Dimension : 2D · **Difficulté :** Intermédiaire

500 disques de rayons aléatoires déposés par gravité dans une boîte.

Paramètre	Valeur
Nombre de particules	500
Rayons	[0.03, 0.08] m
Conteneur	<code>Box2D, lx=4.0, ly=4.0</code>
Seed	42 (reproductible)
Groupe	<code>depot_box</code>

4.2 Boucle géométrique — Cercle

Dimension : 2D · **Difficulté :** Débutant

12 disques disposés en cercle (rayon 2.0 m) autour d'un avatar modèle.

5. Avancé — Contact et mécanismes

5.1 Tambour rotatif

Dimension : 2D · **Difficulté :** Avancé

Disque creux (`is_hollow=True`) entraîné en rotation constante, contenant un dépôt granulométrique.

Élément	Détail
Tambour	<code>RIGID_DISK</code> , $r = 2.2$ m, <code>is_hollow=True</code> → contacteur <code>xKSID</code>
DOF tambour	translation bloquée (<code>component=[1,2]</code>) + rotation entraînée (<code>component=3, ct=0.5 rad/s</code>)
Dépôt interne	<code>Drum2D</code> , $r = 2.0$ m ($<$ rayon tambour), 200 particules, [0.05, 0.09] m
Loi	<code>IQS_CLB</code> , friction = 0.45
Post-pro	<code>COORDINATION NUMBER</code>

Point clé : le contacteur `xKSID` (paroi cylindrique **intérieure**) est le même mécanisme que celui utilisé par le conteneur `Drum2D` de la génération granulométrique standard — voir aussi Roulement à billes qui réutilise ce même principe pour la bague extérieure.

5.2 Décharge en trémie

Dimension : 2D · **Difficulté** : Avancé

Trémie en V construite à partir de deux **roughWall** inclinés, recevant un dépôt granulométrique de 180 disques.

Élément	Détail
Géométrie trémie	top_width=1.6, bottom_width=0.45, height=1.2 m
Parois	2× ROUGH_WALL + rotation DOFOperation(operation_type="rotate", ...) autour du centre propre du mur
Immobilisation	imposeDrivenDof de groupe (hopper_walls)
Dépôt	Box2D , 180 particules, [0.04, 0.07] m
Post-pro	KINETIC ENERGY

5.3 Contact roue/rail (ferroviaire)

Dimension : 3D · **Difficulté** : Avancé

Roue cylindrique roulant sur un rail modélisé en plan rigide, avec un tronçon de **joint de rail** présentant un léger défaut d'alignement.

Élément	Détail
Roue	RIGID_CYLINDER , Ø 920 mm (norme UIC), largeur bandage 135 mm
Rail	RIGID_PLAN , longueur 3.0 m, champignon 70 mm
Joint de rail	second RIGID_PLAN , défaut vertical de 4 mm
DOF roue	guidage latéral bloqué (component=2) + translation longitudinale imposée (component=1, ct=1.5 m/s)
DOF rail/joint	totalement immobilisés (6 DDL RBDY3)
Loi roulement	IQS_CLB , friction = 0.3 (acier/acier sec)
Loi impact joint	RST_CLB , friction = 0.2, rstn=0.3, rstt=0.15
Post-pro	KINETIC ENERGY, TORQUE EVOLUTION (sur la roue)

Mécanismes illustrés : deux lois de contact **distinctes** appliquées au même couple de types de contacteurs (**CYLND/PLANx**) selon la couleur de l'antagoniste — permet de représenter un comportement différencié (roulement continu vs impact localisé) sans dupliquer les avatars.

5.4 Frein à disque de vélo

Dimension : 3D · **Difficulté** : Avancé

Disque de frein serré par deux plaquettes d'étrier ; c'est le frottement qui freine réellement la roue.

Élément	Détail
Disque	RIGID_CYLINDER , Ø 160 mm, épaisseur 1.8 mm
Plaquettes	2× RIGID_PLAN , 34 × 20 × 8 mm, fermeture du piston à 5 mm/s

Élément	Détail
Moyeu	translation + tangage/lacet bloqués, rotation propre libre
Condition initiale	<code>imposeInitValue(component=6, value=ω)</code> — roue à 25 km/h
Loi de contact	<code>IQS_CLB</code> , friction = 0.40 (semi-métallique / inox à sec)
Post-pro	<code>KINETIC ENERGY</code> , <code>DISSIPATED ENERGY</code> , <code>TORQUE EVOLUTION</code>

Point de fidélité physique : contrairement à un simple entraînement, la rotation du disque est posée comme **condition initiale** (`imposeInitValue`) et non comme vitesse imposée en continu (`imposeDrivenDof`) — sans cela, le frottement des plaquettes n'aurait aucun effet observable et l'exemple ne représenterait pas un freinage.

5.5 Roulement à billes (coupe 2D)

Dimension : 2D · **Difficulté** : Avancé

Coupe transversale d'un roulement rainuré à billes type 608.

Élément	Détail
Bague extérieure	<code>RIGID_DISK</code> creux (<code>is_hollow=True</code>), $r \approx 9.5$ mm, fixe
Bague intérieure	<code>RIGID_DISK</code> plein, $r \approx 5.0$ mm, rotation pure entraînée (~300 tr/min)
Billes	7× <code>RIGID_DISK</code> libres (aucun DOF imposé), calées dans l'entrefer
Loi	<code>IQS_CLB</code> , friction = 0.05 (roulement lubrifié)
Post-pro	<code>KINETIC ENERGY</code> , <code>COORDINATION NUMBER</code> , <code>VIOLATION EVOLUTION</code>

Limitation documentée : LMGC90_GUI ne supporte les disques creux (`is_hollow`) qu'en 2D — le roulement est donc représenté en coupe transversale plutôt qu'en 3D torique complet (même contrainte que Tambour rotatif). Les billes ne reçoivent aucune vitesse imposée : leur mouvement résulte uniquement du contact avec les deux pistes, cohérent avec le principe physique d'un roulement réel.

6. Avancé — Déformable

6.1 Corps déformable sur sol rigide

Dimension : 2D · **Difficulté** : Avancé

Rectangle déformable (maillage triangulaire, matériau élastique) chutant sur un mur rigide.

Élément	Détail
Matériau déformable	<code>ELAS1</code> , ELAS, young=70 GPa, $\nu=0.3$
Maillage	<code>pre.buildMesh2D("2T3", ...)</code> , 6×3 éléments, rectangle 1.0×0.4 m
Modèle EF	<code>femxx</code> , élément <code>T3xxx</code>
Sol	<code>SMOOTH_WALL</code> rigide
Loi	<code>GAP_SGR_CLB</code> , friction = 0.3 (rigide/déformable)

7. Synthèse

7.1 Scène composite

Dimension : 2D · Difficulté : Avancé

L'exemple le plus complet du registre : combine presque tous les mécanismes dans un seul projet.

Catégorie	Contenu
Avatars	disque, jonc, polygone (losange), cluster, mur de briques, rampe inclinée
Matériaux	3 (brick , TDURx , steel), densités différentes
Lois de contact	IQS_CLB (frottement pur), RST_CLB (restitution), IQS_MOHR_DS_CLB (cohésion + frottement)
Visibilité	tables croisées par paire de couleurs (pas seulement même-couleur)
Variables dynamiques	11 variables interdépendantes (state.dynamic_vars), consultables via Outils → Variables dynamiques après chargement

Intérêt pédagogique : montre comment les variables dynamiques peuvent piloter toute la géométrie d'une scène (**site_width**, **joint_thickness**, **disk_spacing** = **disk_radius** * **spacing_factor**, ...), en réutilisant **SafeEvaluator** / **build_eval_context** exactement comme le ferait un champ de formulaire dans l'UI.

8. Tableau récapitulatif complet

Ce tableau liste tous les exemples que vous pourriez ouvrir/sauvegarder et visualiser.

ID	Titre	Dim.	Difficulté	Tags principaux
falling_disks	Chute de disques 2D	2D	Débutant	avatar, contact, boucle
sphere_stack	Empilement de sphères 3D	3D	Débutant	avatar, 3d
masonry_wall	Mur de maçonnerie	2D	Intermédiaire	maconnerie, groupe
granulo_deposit	Dépôt granulométrique	2D	Intermédiaire	granulo, masse
circle_loop	Boucle géométrique — Cercle	2D	Débutant	boucle, groupe
deformable_drop	Corps déformable sur sol rigide	2D	Avancé	deformable, fem
cohesive_wall	Mur avec liaisons cohésives (CZM)	2D	Avancé	contact, czm
dumbbell_avatar	Avatar composite — haltère	2D	Intermédiaire	avatar_vide, contacteurs
for_loop_ramp	Boucle For — rampe de rayons	2D	Intermédiaire	boucle, for
dof_conditions	Conditions aux limites DOF	2D	Intermédiaire	dof, conditions_limites
couette_shear	Cisaillement en cellule de Couette	2D	Avancé	granulo, couette

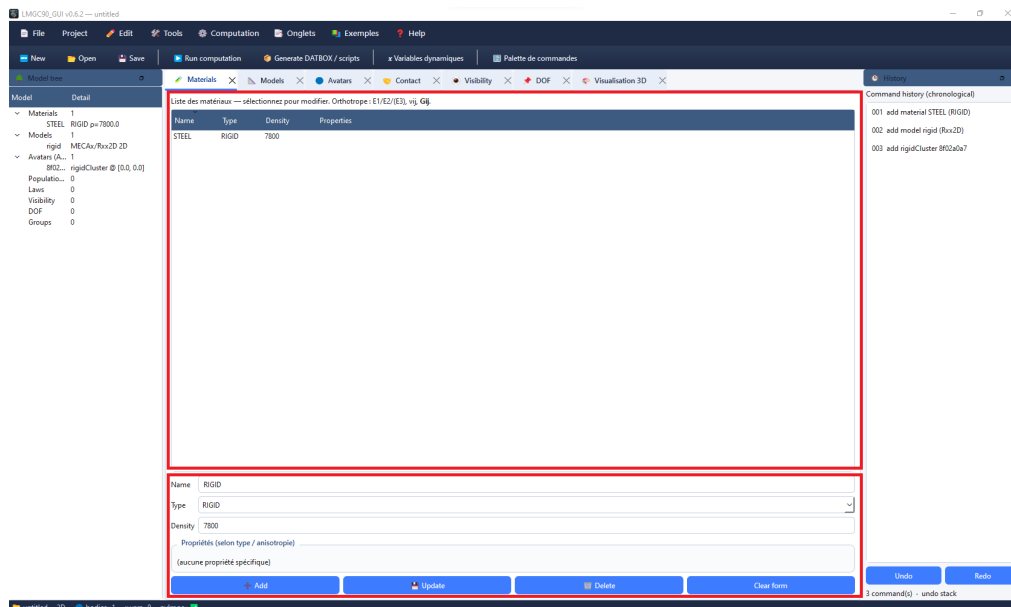
ID	Titre	Dim.	Difficulté	Tags principaux
hopper_discharge	Décharge en trémie	2D	Avancé	factory, granulo, dof
cable_pendulum	Pendule à câble	2D	Avancé	contact, dof, point_point
deformable_impact	Impact déformable (contacteur complet)	2D	Avancé	deformable, contact
l_shaped_wall	Structure en L + granulo	2D	Avancé	maconnerie, granulo, dof
silo_factory	Factory en silo	2D	Avancé	factory, contact, postpro
rotating_drum	Tambour rotatif	2D	Avancé	dof, granulo, rotation
biaxial_compression	Compression biaxiale	2D	Avancé	dof, granulo, essai_mecanique
hexagon_packing	Pavage hexagonal	2D	Intermédiaire	avatar, polygone
cluster_pile	Empilement de clusters	2D	Intermédiaire	avatar, cluster
avalanche_slope	Avalanche sur pente inclinée	2D	Avancé	dof, granulo, pente
wheel_rail_contact	Contact roue/rail (ferroviaire)	3D	Avancé	contact, 3d, ferroviaire, dof, postpro
disc_brake	Frein à disque de vélo	3D	Avancé	contact, 3d, frein, dof, postpro, energie
ball_bearing	Roulement à billes (coupe 2D)	2D	Avancé	contact, roulement, dof, postpro
composite_scene	Scène composite — synthèse complète	2D	Avancé	synthese, variables, contact, avatar

Créer un matériau

L'onglet **Materials** contient la liste des matériaux du projet et un formulaire de création/modification. Les propriétés affichées changent en fonction du type sélectionné ; il ne s'agit pas d'un champ texte libre.

Tous les onglets dans LMGC90_studio sont composés de deux sections :

- **_Liste_** : liste tous vos éléments créés;
- **_Formulaire_** : Renseignez les paramètres de vos éléments à travers des zones de textes, des listes déroulantes ou des cases à cocher.



../captures/onglet_sections.png

Ajouter un matériau

1. Ouvrez **Onglets** → **Ouvrir** → **Materials** (**Ctrl+1**).
2. Saisissez un **Name** de cinq caractères maximum. Un nom peut être suggéré automatiquement selon le type.
3. Choisissez **Type**.
4. Renseignez **Density** si le champ est activé. La densité est utilisée par LMGC90 pour les types qui la prennent en charge.
5. Complétez les champs dans **Propriétés (selon type / anisotropie)**. Les choix et les champs numériques sont adaptés au matériau choisi.
6. Cliquez sur **Add**. Une erreur de validation apparaît si un nom ou une propriété requise est invalide.
7. Sélectionnez une ligne du tableau pour recharger ses valeurs. Modifiez-les puis cliquez sur **Update**.
8. Pour supprimer un matériau, sélectionnez-le et cliquez sur **Delete**. Si d'autres objets le référencent, corrigez d'abord ces dépendances.
9. Cliquez sur **Clear form** pour préparer une nouvelle entrée.

Les valeurs initiales servent de point de départ, pas de caractérisation automatique du matériau réel. Vérifiez unités et constantes physiques avant le calcul.

Paramètres proposés par type

LMGC90_GUI propose **10 types** de matériaux, correspondant aux types acceptés par `pre.material(materialType=...)` dans `pylmgc90`.

Note : les types marqués *_(avancé)_* — **DISCRETE**, **USER_MAT**, **EXTERNAL** — ne disposent pas de suggestion automatique dans l'interface. Leurs paramètres doivent être renseignés manuellement.

Type	Champs principaux de l'interface	Notes
RIGID	Aucun champ spécifique	Utiliser Density .
ELAS	<code>elas</code> , <code>anisotropy</code> , <code>young</code> , <code>nu</code> , <code>G</code> ou constantes directionnelles	Seul type où l'interface propose orthotropic .
ELAS_DILA	<code>elas</code> , isotropie, <code>young</code> , <code>nu</code> , <code>dilatation</code> , <code>T_ref_meca</code>	Pas de <code>G</code> ni de champs orthotropes.
VISCO_ELAS	<code>elas</code> , isotropie, <code>young</code> , <code>nu</code> , <code>viscous_model</code> , <code>viscous_young</code> , <code>viscous_nu</code>	Pas de champ <code>viscosity</code> ni de <code>G</code> .
ELAS_PLAS	<code>young</code> , <code>nu</code> , <code>critere</code> , <code>isoh</code> , <code>iso_hard</code> , <code>isoh_coeff</code> , <code>cinh</code> , <code>visc</code>	Les choix sont fournis par des listes contrôlées.
THERMO_ELAS	<code>young</code> , <code>nu</code> , <code>dilatation</code> , <code>T_ref_meca</code> , <code>conductivity</code> , <code>specific_capacity</code>	<code>conductivity</code> et <code>specific_capacity</code> acceptent une valeur ou field .
PORO_ELAS	<code>young</code> , <code>nu</code> , <code>hydro_cpl</code> , <code>conductivity</code> , <code>specific_capacity</code>	<code>conductivity</code> et <code>specific_capacity</code> acceptent une valeur ou field .
DISCRETE	<code>masses</code> , <code>stiffnesses</code> , <code>viscosities</code>	Saisir un vecteur 2D ou 3D ; le champ densité est désactivé.
USER_MAT	<code>file_mat</code>	Indiquer le chemin/nom de la loi matériau selon la configuration LMG90.
EXTERNAL	Aucun champ de propriété	Le champ densité est désactivé ; le comportement est géré à l'extérieur.

Tableau des usages typiques

Type	Applications typiques	Exemples concrets	Domaines
RIGID	Méthode des éléments discrets (DEM)	Empilements de grains, écoulements granulaires, assemblages de particules	Génie civil, pharmacie, agroalimentaire
ELAS	Structures en régime élastique	Bâtiments, ponts, pièces mécaniques, structures métalliques	Génie civil, mécanique
ELAS_DILA	Contraintes thermiques unilatérales	Structures soumises à des variations de température, dilatation différentielle	Bâtiment, mécanique, électronique
VISCO_ELAS	Matériaux à comportement visqueux	Polymères, asphalte, matériaux amortissants, joints d'étanchéité	Routes, automobile, aéronautique
ELAS_PLAS	Déformations plastiques permanentes	Formage des métaux, impact, endommagement, usinage	Métallurgie, automobile, aéronautique
THERMO_ELAS	Couplage thermo-mécanique complet	Dissipation thermique, chocs thermiques, freinage	Électronique, automobile, nucléaire

Type	Applications typiques	Exemples concrets	Domaines
PORO_ELAS	Milieux poreux saturés	Consolidation de sols, réservoirs pétroliers, aquifères, stockage CO ₂	Géotechnique, hydrogéologie, pétrole
DISCRETE	Systèmes masse-ressort-amortisseur	Isolateurs sismiques, suspensions, liaisons élastiques discrètes	Génie parasismique, automobile
USER_MAT	Lois de comportement sur mesure	Matériaux spécifiques, lois issues de l'expérience	Recherche, matériaux innovants
EXTERNAL	Couplage avec un code externe	Interface avec d'autres logiciels de simulation	Simulation multi-physique

Élastique isotrope et orthotrope

Pour **ELAS**, choisissez **isotropic** pour renseigner **Young** et **Poisson**. Pour une loi orthotrope, choisissez **orthotropic** : les propriétés isotropes seront remplacées; **G** est également proposé.

- En 2D : **young1**, **young2**, **nu12**, **G12**.
- En 3D : **young1**, **young2**, **young3**, **nu12**, **nu13**, **nu23**, **G12**, **G13**, **G23**.

Les champs exclusivement 3D sont masqués dans un projet 2D. Les types **ELAS_DILA**, **VISCO_ELAS**, **ELAS_PLAS**, **THERMO_ELAS** et **PORO_ELAS** restent isotropes dans ce formulaire : ne tentez pas de leur ajouter des propriétés orthotropes.

Options visqueuses

Pour **VISCO_ELAS**, utilisez **viscous_model** (**none** ou **KelvinVoigt**) puis les propriétés **viscous_young** et **viscous_nu** si le modèle visqueux est actif. Les anciennes propriétés nommées **eta** ou **viscosity** ne sont pas des options acceptées par cette version de pylmgc90 et ne doivent pas être saisies.

Élasto-plasticité

ELAS_PLAS propose notamment le critère **Von-Mises** ou **none**, les modèles d'écrouissage isotrope/cinématique et leur paramétrage. Choisissez des valeurs cohérentes entre **isoh**, **cinh** et **visc** ; pylmgc90 valide les choix au moment de la matérialisation.

Paramètres numériques particuliers

- Les valeurs numériques sont saisies dans les champs prévus. Les contrôles de l'interface exposent les propriétés selon le type choisi.
- Pour **THERMO_ELAS** et **PORO_ELAS**, saisissez **field** dans les champs de conductivité/capacité lorsqu'elles sont portées par le modèle éléments finis plutôt que par un scalaire.
- Pour **DISCRETE**, saisissez les composantes séparées par des virgules, par exemple **1, 1** en 2D ou **1, 1, 1** en 3D. Chaque vecteur doit avoir la même dimension que le projet.
- **RIGID** n'a pas de propriété complémentaire ; renseignez la densité.

Exemple : matériau élastique acier

1. Ouvrez **Materials**.
2. Choisissez **ELAS**.
3. Saisissez un nom tel que **STEEL** (5 caractères maximum).
4. Gardez **isotropic**.

-
5. Renseignez **Density** à environ 7850 kg/m³, **Young** à 2.1e11 Pa et **Poisson** à 0.3.
 6. Cliquez sur **Add**.
 7. Créez ensuite un modèle mécanique et associez ce matériau à un avatar.

Mise à jour et suppression

Sélectionnez le matériau dans la liste avant de cliquer sur **Update**, cela chargera toutes les informations de votre matériau dans le formulaire. Vérifiez le nom : si vous saisissez un nom déjà utilisé, ajoutez un nouveau matériau avec **Add** ou mettez à jour explicitement l'entrée sélectionnée.

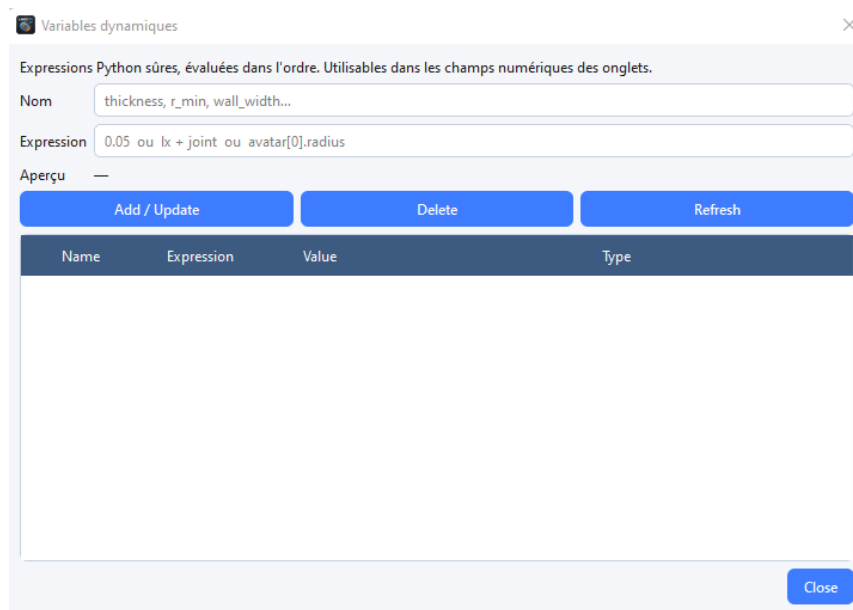
Remarque : La suppression n'est pas un moyen de renommer ou de remplacer automatiquement toutes les références. Réassignez d'abord les avatars concernés à un autre matériau si l'application signale une dépendance.

Variables dynamiques

Les champs numériques prenant en charge les expressions peuvent utiliser les variables du projet. Ouvrez **Tools** → **Variables dynamiques...** (Ctrl+V) pour les créer, voir leur valeur résolue et les modifier. Le dialogue et les limites d'expression sont détaillés dans Variables dynamiques.

Variables dynamiques

Les variables dynamiques enregistrent des expressions réutilisables dans les champs numériques de l'interface qui prennent en charge les expressions. Le dialogue affiche leur expression, leur valeur résolue et leur type.



../captures/variables.png

Ajouter une variable

1. Ouvrez **Tools** → **Variables dynamiques...** (**Ctrl+V**).
2. Saisissez un **Nom** constitué d'un identifiant valide, par exemple **r_min** ou **epaisseur** (sans espace et sans commencer par un chiffre).
3. Saisissez l'**Expression**, par exemple **0.05** ou **largeur + joint**.
4. Consultez **Aperçu**. Le dialogue indique la valeur obtenue ou l'erreur de résolution.
5. Cliquez sur **Add / Update**.
6. Vérifiez l'expression, la valeur et le type dans le tableau.

Modifier, rafraîchir ou supprimer

- Cliquez une ligne du tableau pour charger le nom et l'expression dans le formulaire, puis cliquez **Add / Update** pour enregistrer la modification.
- Cliquez **Refresh** pour recalculer les valeurs affichées.
- Sélectionnez une ligne ou saisissez son nom, puis cliquez **Delete** pour supprimer la variable.
- Cliquez **Close** pour quitter le dialogue.

Les expressions sont évaluées dans l'ordre des variables du projet ; une expression peut s'appuyer sur une variable définie précédemment. Une référence inconnue ou circulaire déclenche une erreur de prévisualisation.

Utiliser une variable dans la scène

1. Définissez la variable dans le dialogue.
2. Ouvrez un onglet qui accepte les expressions numériques, par exemple **Avatars**.
3. Saisissez le nom de la variable dans un champ numérique, par exemple dans un rayon ou une coordonnée.
4. Validez la création ou la modification et corrigez toute erreur signalée.

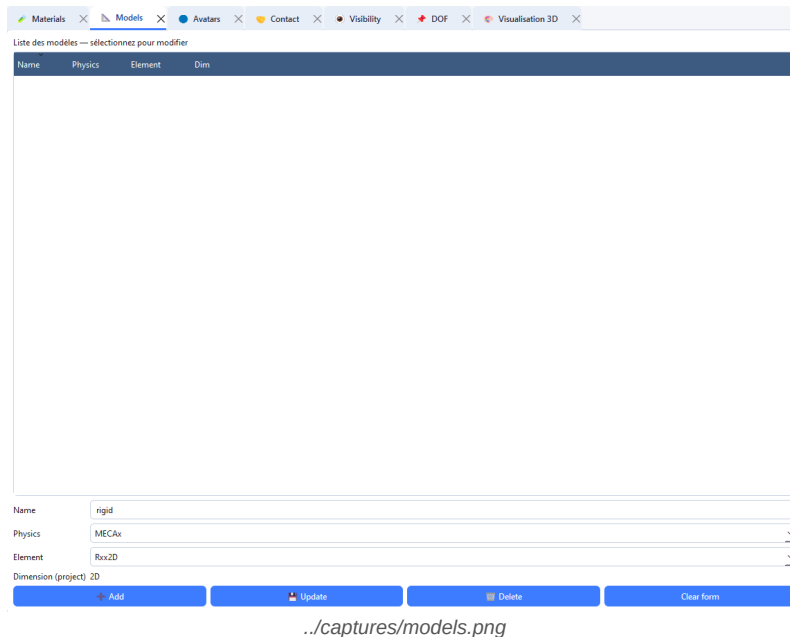
Exemples d'expressions :

Expression	Utilisation possible
0.05	Rayon ou longueur fixe.
diametre / 2	Rayon dérivé d'une dimension définie auparavant.
largeur + joint	Distance composée de paramètres du projet.

Remarque : Le dialogue utilise un évaluateur restreint pour les expressions autorisées. Ce n'est pas un interpréteur Python général : ne comptez pas sur l'accès à tous les modules Python ou à toutes les méthodes d'objets.

Créer un modèle

L'onglet **Models** associe une famille physique et un élément fini à une dimension adapté à vos avatars.



Créer un modèle

1. Ouvrez **Onglets** → **Ouvrir** → **Models** (**Ctrl+2**).
2. Saisissez un **Name** de cinq caractères maximum. Une proposition de nom peut apparaître automatiquement.
3. Choisissez **Physics** : **MECAx**, **THERx**, **POR0x** ou **MULTI**.
4. Choisissez **Element** dans la liste. Elle est actualisée selon la physique et la dimension du projet.
5. Vérifiez **Dimension (project)**. Cette dimension est héritée du projet ; elle ne se règle pas dans le formulaire du modèle.
6. Cliquez sur **Add**.
7. Vérifiez que le modèle apparaît dans le tableau.

Remarque : Un nom déjà utilisé ne crée pas une deuxième entrée : sélectionnez le modèle existant et choisissez **Update** pour le modifier.

Modifier ou supprimer

1. Cliquez sur la ligne du modèle dans la liste.
2. Les champs **Name**, **Physics** et **Element** sont rechargés.
3. Modifiez les valeurs puis cliquez sur **Update**.
4. Pour supprimer le modèle sélectionné, cliquez sur **Delete**. Vérifiez auparavant qu'aucun avatar ou configuration ne le référence.
5. Utilisez **Clear form** pour abandonner la sélection et préparer une nouvelle entrée.

Choisir la physique et l'élément

Les quatre physiques sont toutes implémentées dans LMGC90_GUI.

Code	Nom complet	Description	Matériaux compatibles
MECAX	Mécanique des solides	Déformation, contrainte, dynamique.	RIGID, ELAS, ELAS_DILA, VISCO_ELAS, ELAS_PLAS
THERX	Thermique	Diffusion de chaleur, convection, rayonnement.	THERMO_ELAS
POROX	Poromécanique	Couplage solide / fluide selon la théorie de Biot.	PORO_ELAS
MULTI	Thermo-hydraulique-mécanique (THM)	Couplage complet mécanique + thermique + hydraulique.	PORO_ELAS, THERMO_ELAS

Remarque : La liste déroulante est la référence pour les choix disponibles avec la version installée ; elle se met à jour lorsque vous changez la physique.

Modèle rigide : exemples

Corps rigide 2D

1. Assurez-vous que le projet est en 2D.
2. Dans **Models**, saisissez par exemple **rigid**.
3. Choisissez **MECAX**.
4. Choisissez **Rxx2D**.
5. Cliquez sur **Add**.

Corps rigide 3D

1. Créez ou utilisez un projet 3D.
2. Saisissez un nom court, par exemple **rig3D**.
3. Choisissez **MECAX**, puis **Rxx3D**.
4. Cliquez sur **Add**.

Modèle d'éléments finis

1. Créez un projet de la dimension souhaitée.
2. Dans **Models**, choisissez une physique compatible avec l'analyse envisagée.
3. Sélectionnez l'élément proposé correspondant au type de maillage qui sera construit ou importé.
4. Cliquez sur **Add**.
5. Associez ce modèle à un matériau adapté dans **Materials**, puis choisissez-le lors de la création du corps dans **Avatars** ou dans l'assistant **Deformable**.

Remarque : L'assistant de corps déformable possède des options supplémentaires propres à son parcours de maillage. Elles ne sont pas des champs de l'onglet **Models** : voir Assistant de corps déformable.

Éléments disponibles par physique

MECAx — Éléments mécaniques 2D

Élément	Géométrie	Nœuds	Ordre	Description
Rxx2D	Point	1	—	Corps rigide 2D. Aucune option.
T3xxx	Triangle 3 nœuds	3	1	Triangle linéaire standard.
T3Lxx	Triangle 3 nœuds	3	1	Triangle linéaire enrichi (incompatible).
T6xxx	Triangle 6 nœuds	6	2	Triangle quadratique.
DKTxx	Triangle 3 nœuds	3	1	Triangle de Kirchhoff discret (plaques minces).
Q4xxx	Quadrangle 4 nœuds	4	1	Quadrangle bilinéaire standard.
Q4P0x	Quadrangle 4 nœuds	4	1	Quadrangle bilinéaire + pression constante (quasi-incompressible).
Q8xxx	Quadrangle 8 nœuds	8	2	Quadrangle sérendipité.
Q8Rxx	Quadrangle 8 nœuds	8	2	Quadrangle sérendipité à intégration réduite.
Q9xxx	Quadrangle 9 nœuds	9	2	Quadrangle Lagrange biquadratique.
BARxx	Segment 2 nœuds	2	1	Barre / treillis 1D.
SPRG2	Segment 2 nœuds	2	1	Ressort 2D (discrete=yes__ ajouté automatiquement).

MECAx — Éléments mécaniques 3D

Élément	Géométrie	Nœuds	Ordre	Description
Rxx3D	Point	1	—	Corps rigide 3D. Aucune option.
TE4xx	Tétraèdre 4 nœuds	4	1	Tétraèdre linéaire.
TE4Lx	Tétraèdre 4 nœuds	4	1	Tétraèdre linéaire enrichi (F-bar).
TE10x	Tétraèdre 10 nœuds	10	2	Tétraèdre quadratique.
H8xxx	Hexaèdre 8 nœuds	8	1	Hexaèdre trilineaire.
H20xx	Hexaèdre 20 nœuds	20	2	Hexaèdre sérendipité.
H20Rx	Hexaèdre 20 nœuds	20	2	Hexaèdre sérendipité à intégration réduite.
PRI6x	Prisme 6 nœuds	6	1	Prisme linéaire.
SHB6x	Prisme 6 nœuds	6	1	Prisme solide-coque SHB6.
PRI15	Prisme 15 nœuds	15	2	Prisme quadratique.
BARxx	Segment 2 nœuds	2	1	Barre / treillis 1D.

Élément	Géométrie	Nœuds	Ordre	Description
SPRG3	Segment 2 nœuds	2	1	Ressort 3D (discrete=yes__ ajouté automatiquement).

Remarque : Si vous devez changer la dimension du projet, passez par **Project** → **Set dimension...** et examinez l'avertissement avant de confirmer. Le changement ne convertit pas automatiquement les modèles ou avatars existants.

THERx — Éléments thermiques 2D

Élément	Nœuds	Ordre	Options thermiques spécifiques
Rxx2D	1	—	aucune
T3xxx	3	1	mass_storage, convection, radiation
T6xxx	6	2	mass_storage, convection, radiation
DKTxx	3	1	mass_storage, convection, radiation
Q4xxx	4	1	mass_storage, convection, radiation
Q4P0x	4	1	mass_storage, convection, radiation
Q8xxx	8	2	mass_storage, convection, radiation
Q8Rxx	8	2	mass_storage, convection, radiation
SPRG2	2	1	mass_storage uniquement
S2xth	2	1	mass_storage uniquement — segment thermique 1D

THERx — Éléments thermiques 3D

Élément	Nœuds	Ordre	Options thermiques spécifiques
Rxx3D	1	—	aucune
TE4xx	4	1	mass_storage, convection, radiation
TE10x	10	2	mass_storage, convection, radiation
H8xxx	8	1	mass_storage, convection, radiation
H20xx	20	2	mass_storage, convection, radiation
H20Rx	20	2	mass_storage, convection, radiation
PRI6x	6	1	mass_storage, convection, radiation

Élément	Nœuds	Ordre	Options thermiques spécifiques
PRI15	15	2	mass_storage, convection, radiation
SPRG3	2	1	mass_storage uniquement

POROX — Éléments poromécaniques (éléments mixtes déplacement-pression)

Élément	Dim.	Géométrie	Nœuds	Ordre	Description
T33xx	2D	Triangle	3	1	Triangle mixte P1/P1.
T63xx	2D	Triangle	6	2	Triangle mixte P2/P1 — satisfait la condition LBB.
Q44xx	2D	Quadrangle	4	1	Quadrangle mixte Q1/Q1.
Q84xx	2D	Quadrangle	8	2	Quadrangle mixte Q2/Q1 — satisfait la condition LBB.
TE44x	3D	Tétraèdre	4	1	Tétraèdre mixte P1/P1.
TE104	3D	Tétraèdre	10	2	Tétraèdre mixte P2/P1 — satisfait la condition LBB.
H88xx	3D	Hexaèdre	8	1	Hexaèdre mixte Q1/Q1.
H208x	3D	Hexaèdre	20	2	Hexaèdre mixte Q2/Q1 — satisfait la condition LBB.

MULTI — Éléments THM 2D et 3D

Mêmes éléments mixtes que POROX, avec en plus H8xxx en 3D pour l'interpolation uniforme des trois champs couplés.

Élément	Dim.	Description
T33xx	2D	Triangle mixte P1/P1.
T63xx	2D	Triangle mixte P2/P1.
Q44xx	2D	Quadrangle mixte Q1/Q1.
Q84xx	2D	Quadrangle mixte Q2/Q1.
TE44x	3D	Tétraèdre mixte P1/P1.
TE104	3D	Tétraèdre mixte P2/P1.
H8xxx	3D	Hexaèdre trilineaire (interpolation uniforme THM).
H88xx	3D	Hexaèdre mixte Q1/Q1.
H208x	3D	Hexaèdre mixte Q2/Q1.

Tableau récapitulatif par usage

Usage	Dimension	Éléments recommandés	Remarque
Corps rigides 2D (DEM)	2D	Rxx2D	Aucune option
Corps rigides 3D (DEM)	3D	Rxx3D	Aucune option
Ressorts / barres 2D	2D	SPRG2, BARxx	discrete=yes__ auto
Ressorts / barres 3D	3D	SPRG3, BARxx	discrete=yes__ auto
Structures 2D — précision standard	2D	Q4xxx, T3xxx	Rapide, adapté aux maillages structurés
Structures 2D — géométrie complexe	2D	T6xxx, Q8xxx	Maillages automatiques non structurés
Plaques minces 2D	2D	DKTxx	Formulation Kirchhoff
Quasi-incompressibilité 2D	2D	Q4P0x	Pression constante par élément
Structures 3D — maillage structuré	3D	H8xxx, H20xx	Hexaèdres recommandés
Structures 3D — maillage automatique	3D	TE10x, TE4xx	Tétraèdres adaptatifs
Coques épaisses 3D	3D	SHB6x, PRI6x	Solidescoques
Thermique 2D	2D	Q4xxx, T3xxx	Physique THERx
Thermique 3D	3D	H8xxx, TE4xx	Physique THERx
Segment thermique 1D	2D / 3D	S2xth	Physique THERx uniquement
Poro-mécanique 2D (Biot)	2D	T63xx, Q84xx	LBB satisfaite — recommandé
Poro-mécanique 3D (Biot)	3D	TE104, H208x	LBB satisfaite — recommandé
THM couplé 3D	3D	H8xxx, TE104	Physique MULTI

Tableau de compatibilité matériau / physique

Matériau	MECAx	THERx	PORox	MULTI
RIGID	■	■	■	■
ELAS	■	■	■	■
ELAS_DILA	■	■	■	■
VISCO_ELAS	■	■	■	■
ELAS_PLAS	■	■	■	■
THERMO_ELAS	■	■	■	■
PORO_ELAS	■	■	■	■
DISCRETE	■	■	■	■
USER_MAT	■	■	■	■
EXTERNAL	■	■	■	■

Tableau de compatibilité matériau / option material

Matériau	elas__	elasd	J2iso	J2mix	kvisc
RIGID	■	—	—	—	—

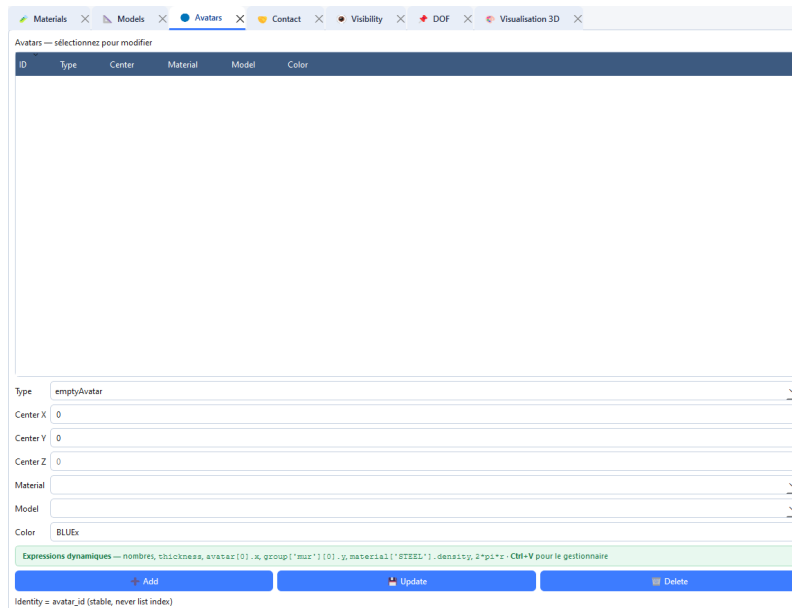
Matériau	elas_	elasd	J2iso	J2mix	kvisc
ELAS	■	—	—	—	—
ELAS_DILA	■	■	—	—	—
VISCO_ELAS	—	—	—	—	■
ELAS_PLAS	—	■	■	■	—
THERMO_ELAS	■	■	—	—	—

Chapitre 7

Avatars — Corps rigides

Créer et modifier un avatar

Un avatar est un corps de la scène. Il référence un matériau et un modèle, possède un centre et une couleur, puis des paramètres géométriques propres à son type. Créez d'abord le matériau et le modèle.



../captures/avatars.png

Créer un avatar

1. Ouvrez **Onglets** → **Ouvrir** → **Avatars** (**Ctrl+3**).
2. Choisissez **Type**. Les types proposés sont adaptés à la dimension du projet.
3. Renseignez **Center X** et **Center Y**. En 3D, renseignez aussi **Center Z** ; ce champ est inactif en 2D.
4. Choisissez **Material** et **Model** dans les listes.
5. Renseignez **Color** avec le code de couleur LMGC90 souhaité (par exemple **BLUEx**, **REDxx** ou **GRAYx**).
6. Complétez **Type parameters**. Les champs changent automatiquement lorsque le type d'avatar change.
7. Cliquez sur **Add**. Une boîte d'avertissement précise les références manquantes ou paramètres invalides.
8. Actualisez le visualiseur 3D si vous souhaitez contrôler le placement.

Types et paramètres géométriques

Les noms de paramètres sont ceux présentés dans **Type parameters**.

Formes 2D

Type	Paramètres spécifiques	
rigidDisk	r, is_Hollow	
rigidDiscreteDisk	r	

Type	Paramètres spécifiques	
rigidJonc	axe1, axe2	
rigidPolygon	generation_type, nb_vertices, radius, vertices	
rigidOvoidPolygon	radius, vertices	
rigidCluster	r, nb_disk	
roughWall	Mur rugueux	l, r, nb_vertex
fineWall	Mur fin	l, r, nb_vertex
smoothWall	Mur lisse	l, h, nb_polyg
granuloRoughWall	Mur granulaire	l, rmin, rmax, nb_vertex

Formes 3D

Type	Paramètres spécifiques
rigidSphere	r, is_Hollow
rigidCylinder	r, h, is_Hollow
rigidPlan	axe1, axe2, axe3
rigidPolyhedron	generation_type, nb_vertices, radius, vertices
roughWall3D	lx, ly, lz
granuloRoughWall3D	lx, ly, lz, rmin, rmax

`emptyAvatar` et `mesh` apparaissent également dans la liste. Leurs procédures sont détaillées dans Avatar vide et Corps déformable ou Assistant des déformables.

Polygones et sommets personnalisés

Pour un polygone ou un polyèdre régulier, gardez `generation_type` à `regular`, puis renseignez le nombre de sommets et le rayon. Pour une géométrie explicite, saisissez les coordonnées dans `vertices`, séparées par des points-virgules :

```
x1,y1; x2,y2; x3,y3
```

En 3D, chaque sommet doit contenir ses coordonnées `x,y,z`. Les sommets doivent former une géométrie valide et contenir au moins trois points en 2D ou quatre en 3D.

Expressions et dimension

Les champs numériques de centre et plusieurs paramètres acceptent une valeur numérique ou une expression du projet. Par exemple, un rayon peut réutiliser une variable définie dans **Tools** → **Variables dynamiques....** Une expression non résolue provoque un avertissement au moment de l'ajout.

En 2D, le centre contient deux coordonnées. En 3D, il en contient trois. Utilisez un modèle de la même dimension ; les types rigides doivent être associés à l'élément rigide de cette dimension (`Rxx2D` ou `Rxx3D`).

Modifier un avatar

1. Sélectionnez la ligne de l'avatar dans la liste supérieure.
2. Les champs communs et les paramètres disponibles pour son type se rechargent.

3. Modifiez les valeurs nécessaires.
4. Cliquez sur **Update**.

Supprimer un avatar

Sélectionnez l'avatar puis cliquez sur **Delete**. Vérifiez les groupes, conditions DOF et autres règles qui pouvaient le cibler. La suppression de l'avatar ne signifie pas nécessairement que toutes les références fonctionnelles du projet ont été réinterprétées.

Couleur et contact

La couleur sert aussi à sélectionner des corps dans les tables **Visibility** et les opérations DOF ciblées par couleur. Gardez un code de couleur cohérent entre l'avatar, son contacteur et la table de visibilité. L'ajout de formes de contact supplémentaires est documenté dans Contacteurs.

Exemples de couleurs que vous pourriez utiliser :

Code	Couleur
BLUEX	Bleu
REDXX	Rouge
VERTX	Vert
JAUNX	Jaune
GRAYX	Gris
BLACX	Noir
WHITX	Blanc
ORANX	Orange
CYANX	Cyan
MAGEX	Magenta
VIOLX	Violet
ROSEX	Rose

Types d'avatars 2D

1. rigidDisk — Disque rigide 2D

Corps circulaire rigide 2D. C'est le type le plus courant pour les simulations granulaires.

Paramètre	Description	Exemple
r (rayon)	Rayon du disque (m).	0.1
is_hollow	Si coché, crée un disque creux (is_Hollow=True). Utile pour les anneaux rigides.	case à cocher

rigidJonc — Jonc / Ellipse rigide 2D

Corps elliptique rigide 2D. Défini par deux demi-axes.

Paramètre	Description	Exemple
<code>axe1</code>	Demi-axe principal (m) — longueur.	<code>0.15</code>
<code>axe2</code>	Demi-axe secondaire (m) — largeur.	<code>0.05</code>

rigidPolygon — Polygone rigide 2D

Corps polygonal rigide 2D. Trois modes de génération disponibles selon `generation_type`.

Paramètre	Description	Valeurs
<code>generation_type</code>	Mode de génération de la forme.	<code>regular</code> · <code>full</code> · <code>bevel</code>
<code>nb_vertices</code>	Nombre de sommets (pour <code>regular</code> et <code>full</code>).	3 à 20
<code>radius</code>	Rayon du cercle circonscrit (m) — utilisé pour <code>regular</code> . Non utilisé pour <code>full</code> et <code>bevel</code> .	<code>0.1</code>
<code>vertices</code>	Liste explicite de sommets <code>[[x1, y1], [x2, y2], ...]</code> — utilisé pour <code>full</code> et <code>bevel</code> .	<code>[[-0.1, -0.1], [0.1, -0.1], [0., 0.1]]</code>

Modes de génération :

- `regular` : polygone régulier (tous les côtés égaux). Défini par `nb_vertices` et `radius` (rayon du cercle circonscrit).
- `full` : polygone quelconque à partir d'une liste de sommets explicites. Le rayon n'est pas utilisé.
- `bevel` : polygone avec chanfreinage automatique des angles pour éviter les singularités de contact.

rigidOvoidPolygon — Ovoïde rigide 2D

Corps ovoïde (ellipse polygonale) rigide 2D. Approximation polygonale d'une ellipse.

Paramètre	Description	Exemple
<code>ra</code>	Demi-axe principal (m).	<code>0.15</code>
<code>rb</code>	Demi-axe secondaire (m).	<code>0.08</code>
<code>nb_vertices</code>	Nombre de sommets de l'approximation polygonale.	<code>20</code>

rigidDiscreteDisk — Disque discret rigide 2D

Corps circulaire rigide 2D à cinématique discrète. Même géométrie qu'un `rigidDisk`, mais avec un contacteur de type `xKSID` (disque discret). Utilisé dans certains modèles discrets avancés.

Paramètre	Description	Exemple
<code>r</code> (rayon)	Rayon du disque (m).	<code>0.1</code>

rigidCluster — Cluster de disques rigides 2D

Corps rigide 2D composé de plusieurs disques liés rigidement. Permet de créer des formes non convexes complexes.

Paramètre	Description	Exemple
<code>r</code>	Rayon de chaque disque dans le cluster (m).	<code>0.05</code>
<code>nb_vertices</code> (<code>nb_disk</code>)	Nombre de disques dans le cluster.	<code>4</code>

Le paramètre est nommé `nb_disk` dans le script `pylmgc90` généré (pas `nb_vertices`).

roughWall — Mur rugueux 2D

Paroi 2D rugueuse composée de disques alignés. Utilisé pour les parois confinantes avec rugosité géométrique.

Paramètre	Description	Exemple
<code>l</code>	Longueur totale du mur (m).	<code>2.0</code>
<code>r</code>	Rayon des disques constituant la rugosité (m).	<code>0.05</code>
<code>nb_vertex</code>	Nombre de disques le long du mur. Défaut : <code>10</code> .	<code>20</code>

fineWall — Mur fin 2D

Paroi 2D fine composée de disques très petits. Mêmes paramètres que `roughWall` mais avec une rugosité plus fine.

Paramètre	Description	Exemple
<code>l</code>	Longueur totale du mur (m).	<code>2.0</code>
<code>r</code>	Rayon des disques (m). Typiquement très petit (0,001 à 0,01).	<code>0.005</code>
<code>nb_vertex</code>	Nombre de disques. Défaut : <code>10</code> .	<code>50</code>

smoothWall — Mur lisse 2D

Paroi 2D lisse définie par une demi-largeur et une hauteur. Contacteur `CLxxx` (surface continue).

Paramètre	Description	Exemple
<code>l</code>	Demi-longueur du mur (m) — la longueur totale est $2 \times l$.	<code>1.0</code>
<code>h</code>	Demi-hauteur (épaisseur) du mur (m).	<code>0.01</code>
<code>nb_polyg</code>	Nombre de segments polygonaux. Défaut : <code>10</code> .	<code>20</code>

granuloRoughWall — Mur granulaire rugueux 2D

Paroi 2D avec rugosité aléatoire générée par distribution granulométrique de disques.

Paramètre	Description	Exemple
<code>l</code>	Longueur totale du mur (m).	<code>2.0</code>
<code>rmin</code>	Rayon minimal des disques (m).	<code>0.01</code>

Paramètre	Description	Exemple
<code>rmax</code>	Rayon maximal des disques (m).	<code>0.05</code>
<code>nb_vertex</code>	Nombre de disques. Défaut : <code>10</code> .	<code>30</code>

Types d'avatars 3D

`rigidSphere` — Sphère rigide 3D

Corps sphérique rigide 3D. Équivalent 3D du `rigidDisk`.

Paramètre	Description	Exemple
<code>r</code> (rayon)	Rayon de la sphère (m).	<code>0.1</code>
<code>is_hollow</code>	Si coché, crée une Sphère creuse (<code>is_Hollow=True</code>).	case à cocher

`rigidPlan` — Plan rigide 3D

Surface plane rigide 3D. Défini par trois vecteurs directeurs formant un repère local.

Paramètre	Description	Exemple
<code>axe1</code>	Vecteur du premier axe du plan (direction X locale).	<code>[1.0, 0.0, 0.0]</code>
<code>axe2</code>	Vecteur du deuxième axe du plan (direction Y locale).	<code>[0.0, 1.0, 0.0]</code>
<code>axe3</code>	Normale au plan (direction Z locale).	<code>[0.0, 0.0, 1.0]</code>

Les trois vecteurs doivent former une base orthonormée directe.

`rigidCylinder` — Cylindre rigide 3D

Corps cylindrique rigide 3D.

Paramètre	Description	Exemple
<code>r</code> (rayon)	Rayon du cylindre (m).	<code>0.1</code>
<code>h</code>	Hauteur (longueur axiale) du cylindre (m). Défaut : <code>1.0</code> si absent.	<code>0.5</code>
<code>is_hollow</code>	Si coché, crée un cylindre creux (<code>is_Hollow=True</code>).	case à cocher

`rigidPolyhedron` — Polyèdre rigide 3D

Corps polyédrique rigide 3D. Deux modes de génération disponibles selon `generation_type`.

Paramètre	Description	Valeurs
<code>generation_type</code>	Mode de génération.	<code>regular</code> · <code>vertices</code>
<code>nb_vertices</code>	Nombre de sommets (pour <code>regular</code>).	<code>8</code> (cube), <code>12</code> (icosaèdre)...

Paramètre	Description	Valeurs
<code>radius</code>	Rayon du polyèdre régulier (m) — pour <code>regular</code> .	<code>0.1</code>
<code>vertices</code>	Liste explicite de sommets 3D <code>[[x,y,z],...]</code> — pour <code>vertices</code> .	<code>[[-1,-1,-1], [1,-1,-1], ...]</code>
<code>faces</code>	Connectivité des faces <code>[[i,j,k],...]</code> (dans <code>wall_params</code>) — pour <code>vertices</code> .	<code>[[0,1,2], [2,3,0], ...]</code>

Modes de génération :

- `regular` : polyèdre régulier (similaire à une sphère polygonale). Défini par `nb_vertices` et `radius`.
- `vertices` : polyèdre quelconque défini par une liste explicite de sommets et la connectivité des faces.

roughWall3D — Mur rugueux 3D

Paroi 3D rugueuse composée de sphères alignées sur une surface rectangulaire.

Paramètre	Description	Exemple
<code>lx</code>	Dimension du mur en X (m).	<code>2.0</code>
<code>ly</code>	Dimension du mur en Y (m).	<code>2.0</code>
<code>r</code>	Rayon des sphères de rugosité (m).	<code>0.05</code>

granuloRoughWall3D — Mur granulaire rugueux 3D

Paroi 3D rugueuse avec sphères de tailles aléatoires.

Paramètre	Description	Exemple
<code>lx</code>	Dimension en X (m).	<code>2.0</code>
<code>ly</code>	Dimension en Y (m).	<code>2.0</code>
<code>rmin</code>	Rayon minimal des sphères (m).	<code>0.01</code>
<code>rmax</code>	Rayon maximal des sphères (m).	<code>0.05</code>

Avatar vide

emptyAvatar crée un corps sans forme de contact définie par défaut. Ajoutez ensuite un ou plusieurs contacteurs dans l'onglet **Contactors**. L'ancien onglet indépendant « Empty Avatar » n'existe plus.

Créer le corps vide

1. Créez le matériau et le modèle compatibles avec la dimension du projet.
2. Ouvrez **Avatars** (**Ctrl+3**).
3. Choisissez **emptyAvatar** dans **Type**.
4. Indiquez son centre X/Y et, en 3D, Z.
5. Choisissez **Material** et **Model**, puis indiquez **Color**.
6. Cliquez sur **Add**.

Un avatar vide ne possède pas de champ géométrique supplémentaire dans **Type parameters**. Sans contacteur, il n'a pas de géométrie de contact à dessiner. **visuAvatars** l'ignore alors ; le journal indique combien d'avatars ont été ignorés. Ajoutez au moins un contacteur pour voir sa forme dans le visualiseur natif.

Ajouter ses contacteurs

1. Ouvrez **Onglets** → **Ouvrir** → **Contactors**.
2. Sélectionnez l'avatar vide dans **Select avatar**.
3. Choisissez une forme proposée dans **Shape**. La liste est filtrée selon le type de corps et la dimension.
4. Renseignez les paramètres géométriques affichés pour cette forme (par exemple **byrd** pour le rayon de **DISKx**, ou **axe1/axe2** pour **JONCx**). Le champ **shift** décale le contacteur depuis le centre de l'avatar ; il prend deux coordonnées en 2D et trois en 3D.

Pour **POLYG** et **POLYR**, choisissez **generation_type** : **regular** demande le nombre de sommets et le rayon ; **full** demande les sommets, ainsi que la connectivité triangulaire des faces pour **POLYR**. Les sommets d'un **POLYG** complet doivent être donnés dans le sens anti-horaire.

5. Renseignez **Color**. Utilisez un code de 5 caractères cohérent avec les tables de visibilité à créer.
6. Pour un corps maillé, indiquez éventuellement **Group** pour limiter le contacteur à un groupe d'éléments.
7. Cliquez sur **Add contactor** et vérifiez la ligne ajoutée.
8. Pour retirer une forme, sélectionnez-la et cliquez sur **Remove selected**.

Après ajout ou suppression, cliquez sur **Rafraîchir la scène** dans le viewer 3D. Le bouton **pre.visuAvatars** reconstruit également les corps pylmgc90 depuis le projet ; un contacteur sans ses dimensions requises n'est pas créé.

Avatar mesh

Il est tout à fait possible de créer un avatar déformable afin d'interagir rigid/déformable. Pour cela il faut opter sur l'élément **mesh**.

Corps déformable 2D — mesh_shapes_2d

Ces formes sont destinées à être ajoutées sur un maillage FEM 2D. Elles définissent des contacteurs de surface pour les interactions rigide-déformable.

Forme	Description	Usage
ALpxx	Contacteur ligne pour maçonnerie FEM 2D	Interactions ALpMECAx (CLALp / MECAX)
CLxx	Contacteur ligne continu 2D	Interactions DKMECAx (disque / MECAX)
DISKL	Disque sur nœud FEM 2D	Interaction disque-disque sur maillage
PT2TL	Point de transmission 2D	Couplage nœud-nœud FEM

Corps déformable 3D — mesh_shapes_3d

Forme	Description	Usage
ASpxx	Contacteur surface pour sphères FEM 3D	Interactions SPMECAx (sphère / MECAX 3D)
CSpxx	Contacteur surface continu 3D	Interactions rigide-déformable 3D génériques
PT3Dx	Nœud ponctuel FEM 3D	Couplage nœud-nœud FEM 3D

Remarque : Un même avatar peut porter plusieurs contacteurs de couleurs ou formes distinctes. Chaque couple forme/couleur utilisé dans une interaction doit être correctement repris dans **Visibility**.

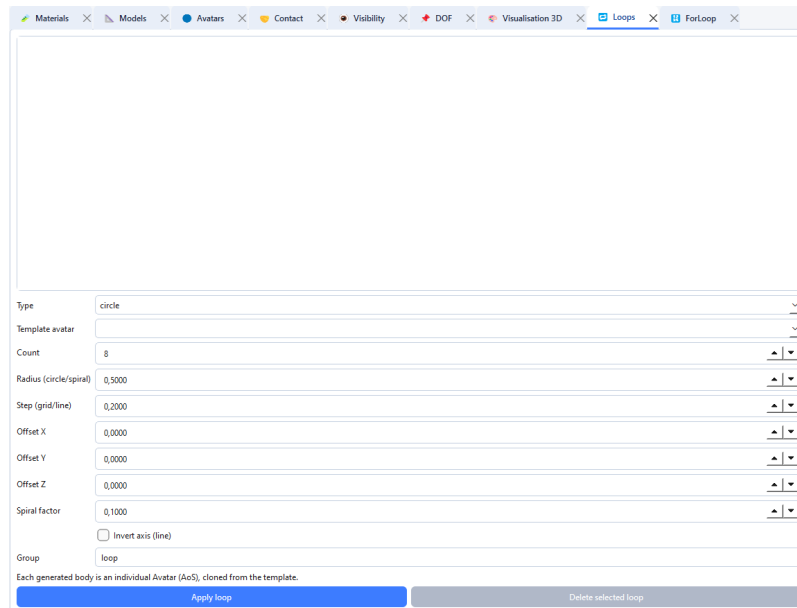
Configurer le contact

1. Créez une loi dans **Contact**.
2. Dans **Visibility**, choisissez le corps, la forme et la couleur du contacteur comme candidat ou antagoniste.
3. Associez la loi et une distance **Alert** adaptée à l'échelle de la scène.
4. Cliquez sur **Add**, puis sur **Vérifier** pour rechercher les couleurs sans table ou les lois non référencées.

Voir Contacteurs, Lois de contact et Tables de visibilité pour les formulaires correspondants.

Boucles géométriques d'avatars

L'onglet **Loops** clone un avatar modèle et répartit ses copies selon une géométrie régulière. Ces boucles sont distinctes des boucles paramétriques multi-cibles de l'onglet ForLoop.



../captures/loops.png

Préparer le modèle

1. Créez un avatar dans **Avatars** avec son matériau, son modèle, sa couleur et ses paramètres géométriques.
2. Vérifiez sa position initiale : elle sert de base au clonage.
3. Ouvrez **Onglets** → **Ouvrir** → **Loops** (Ctrl+4).

Définir la génération

1. Choisissez **Type** : **circle**, **grid**, **line** ou **spiral**.
2. Choisissez **Template avatar**.
3. Définissez **Count**, le nombre de copies à créer.
4. Renseignez les champs adaptés au motif :
 - **circle** : **Radius** détermine le rayon de placement ; utilisez également les offsets pour déplacer le centre de la disposition ;
 - **grid** : **Step** définit l'espacement ; les offsets déplacent la grille ;
 - **line** : **Step** définit l'espacement ; **Invert axis (line)** inverse l'orientation de la ligne ;
 - **spiral** : **Radius** et **Spiral factor** déterminent l'ouverture et la progression, avec les offsets de position.
5. En 3D, renseignez les offsets X, Y et Z nécessaires ; en 2D, l'offset Z n'a pas d'effet géométrique.
6. Renseignez **Group** pour enregistrer les avatars générés dans un groupe.
7. Cliquez sur **Apply loop**.

8. Lisez la confirmation du nombre de corps créés et vérifiez le résultat dans **Avatars** ou le visualiseur 3D après **Rafraîchir la scène**.

Supprimer une boucle

Sélectionnez la boucle dans la liste, puis cliquez sur **Delete selected loop** et confirmez. La boucle et les avatars qu'elle a générés sont retirés ; les autres avatars du projet restent en place. Cette action peut être annulée ou rétablie avec les commandes **Undo** et **Redo**.

Résultat et répétition

Chaque copie est ajoutée au projet comme un avatar indépendant. Appliquer de nouveau une boucle ajoute une nouvelle série ; cela ne remplace pas la génération précédente. Donnez un nom de groupe distinct si vous devez distinguer plusieurs séries.

Les boucles géométriques ne modifient pas directement les propriétés physiques du modèle. Pour faire varier matériaux, modèles, DOF, visibilité ou paramètres granulométriques avec une expression d'itération, utilisez **ForLoop**.

Granulométrie et populations de particules

L'onglet **Granulo** génère une population de particules dont les rayons sont compris entre deux bornes, et les place dans un conteneur. Une population SoA permet de représenter de nombreux grains sans créer une ligne graphique individuelle pour chaque particule.

The screenshot shows the Granulo configuration panel with the following settings:

- N particules:** 200
- Rayon min:** 0,050000
- Rayon max:** 0,150000
- Conteneur:** Box2D (rectangle)
- lx:** 1,000000
- ly:** 1,000000
- Type d'avatar:** rigidDisk
- Matériau:** (empty)
- Modèle:** (empty)
- Couleur (5 car.):** BLUEs
- Groupe (5 car.):** grains
- Seed:** 42
- ☐ Utiliser dépôt py/mgc (engine) si disponible — sinon numpy SoA
- ☐ Distribution seule (rayons) — sans avatars / sans population

Buttons at the bottom: **Générer le dépôt** and **Supprimer sélection**.

../captures/granulo.png

Préparer le projet

1. Créez un matériau compatible avec les grains dans **Materials**.
2. Créez un modèle rigide adapté à la dimension (**MECAx** avec **Rxx2D** ou **Rxx3D**) dans **Models**.
3. Ouvrez **Onglets** → **Ouvrir** → **Granulo** (**Ctrl+6**).

Régler le dépôt

1. Indiquez **N particules**.
2. Renseignez **Rayon min** et **Rayon max**. Le rayon minimum doit être strictement inférieur au maximum.
3. Choisissez le **Conteneur** proposé pour la dimension du projet :
 - 2D : **Box2D**, **Disk2D**, **Drum2D**, **Couette2D** ;
 - 3D : **Box3D**, **Sphere3D**, **Cylinder3D**.
4. Complétez les dimensions du conteneur affichées sous **Conteneur** : **lx**, **ly**, **lz**, **r**, **rint** ou **rext** selon le cas.
5. Choisissez le **Type d'avatar**. En 2D : disque, disque discret ou polygone ; en 3D : sphère ou polyèdre.
6. Sélectionnez **Matériau** et **Modèle**.
7. Renseignez **Couleur (5 car.)** et **Groupe (5 car.)**.
8. Réglez **Seed** pour reproduire le tirage. La valeur **random** demande une graine aléatoire.

Choisir un mode de génération

- **Dépôt NumPy SoA** : laissez **Utiliser dépôt pylmgc (engine)** si disponible décoché. L'application construit une population compacte et l'ajoute au projet.
- **Dépôt pylmgc** : cochez cette option si pylmgc90 est disponible dans l'environnement et que vous souhaitez utiliser ses routines de dépôt.
- **Distribution seule** : cochez **Distribution seule (rayons)** pour générer et enregistrer les rayons sans produire d'avatars ni de population. Dans ce mode, le dépôt pylmgc est désactivé.

Générer et contrôler le résultat

1. Relisez le nombre, les rayons, le conteneur, le type de corps et les références matériau/modèle.
2. Cliquez sur **Générer le dépôt**.
3. Vérifiez le message de confirmation.
4. La liste présente les populations créées sous la forme **[SoA]** avec leur effectif, leur type et leur groupe. Les distributions seules sont repérées par **[dist]**.
5. Pour retirer une population, sélectionnez-la dans la liste puis cliquez sur **Supprimer sélection**.
6. Ouvrez **Visualisation 3D** et cliquez sur **Rafraîchir la scène** pour voir la population.

La suppression proposée concerne une population ; vérifiez la ligne sélectionnée avant de valider.

Dimension et contact

Le type de conteneur doit correspondre à la dimension du projet. Les populations génèrent leurs propres corps rigides/couleurs, mais les interactions nécessitent encore des lois et des tables de visibilité compatibles. Dans **Visibility**, utilisez les couleurs et contacteurs proposés depuis la population ; choisissez la bonne paire candidat/antagoniste et une distance **Alert** cohérente.

Pour répéter une génération avec une variable d'itération, consultez ForLoop. Pour comprendre la représentation des grandes populations, voir Populations de particules (référence technique).

ParticlePopulation — Architecture SoA des particules de masse

Complément au guide développeur. Référence technique (pas un guide d'utilisation graphique). Code : `lmgc90_core/population.py`, `lmgc90_core/io.py`, `lmgc90_core/generate.py`, `lmgc90_engine/granulo.py`, `lmgc90_engine/avatar_factory.py`.

1. Pourquoi SoA ?

Approche	Structure	Cas d'usage	Limite pratique
AoS (<code>project.avatars</code>)	Un objet Avatar par corps	Murs, avatars manuels, déformables, édition unitaire	quelques milliers avant ralentissement de l'UI
SoA (<code>project.populations</code>)	Un ParticlePopulation = deux tableaux numpy	Dépôts granulométriques, grands volumes homogènes	dizaines à centaines de milliers de particules

ParticlePopulation ne remplace pas Avatar : les deux coexistent dans **Project**.

```
Project
■■■ avatars: list[Avatar] # AoS – édition fine
■■■ populations: list[ParticlePopulation] # SoA – volumes
```

```
population_groups: dict[nom → [population_id]]
```

Changement majeur par rapport à l'ancienne version : le SoA n'est plus une option à cocher. `Project.deposit()` produit **toujours** une population (sauf mode « distribution seule »). Il n'y a plus de `particle_populations` dans un `ProjectState`, ni de `_pylmgc_population_bodies` dans le contrôleur.

2. Modèle de données

```
@dataclass
class ParticlePopulation:
    population_id: str # "pop_<uuid>" – un id par population
    avatar_type: AvatarType # homogène
    material_name: str # homogène
    model_name: str # homogène
    color: str # homogène
    origin: AvatarOrigin # GRANULO, LOOP, ...
    dimension: int # 2 ou 3 (= centers.shape[1])
    centers: np.ndarray # (N, dim) float64
    radii: np.ndarray # (N,) float64
    group_name: Optional[str] = None
    extra_params: dict = {} # nb_disk, h, nb_vertices, ...
```

2.1 Types supportés

`types.POPULATION_TYPES` : `RIGID_DISK`, `RIGID_SPHERE`, `RIGID_DISCRETE`, `RIGID_CLUSTER`, `RIGID_CYLINDER`, `RIGID_POLYGON`, `RIGID_POLYHEDRON`.

`extra_params` requis selon le type (validés dans `_validate_extra`) :

Type	extra_params
<code>rigidCluster</code>	<code>nb_disk</code> entier ≥ 2 (défaut 3)
<code>rigidCylinder</code>	<code>h</code> > 0 obligatoire
<code>rigidPolygon</code>	<code>nb_vertices</code> entier ≥ 3
<code>rigidPolyhedron</code>	<code>nb_vertices</code> entier ≥ 4

2.2 Construction validée

Toujours via `create()` :

```
pop = ParticlePopulation.create(
    avatar_type="rigidDisk", # AvatarType ou chaîne
    material_name="TDURx", model_name="rigid",
    centers=centers, radii=radii,
    color="BLUEx",
    origin="granulo",
    group_name="depot",
    population_id=None, # généré si absent
    extra_params=None,
)
```

Erreurs `ValueError` : type non supporté, `centers` pas `(N, 2|3)`, `radii` pas `(N,)`, longueurs différentes, rayon ≤ 0 , `extra_params` invalides.

2.3 Identité des particules

Dérivée, jamais stockée (`ids.py`) :

```
pop.particle_avatar_id(i) # "pop_abc...:42"
parse_particle_id("pop_abc...:42") # ("pop_abc...", 42)
```

Le `population_id` est stable ; l'indice `i` est positionnel (la population est traitée comme un bloc : pas de suppression ou de réordonnancement individuel).

2.4 Vue ponctuelle

```
av = pop.as_avatar_view(i) # construit UN Avatar (avatar_id = id dérivé)
```

À réserver à l'inspection ponctuelle. Ne **jamais** boucler sur toute la population hors cas assumé (le viewer 3D le fait, voir §6).

2.5 Statistiques

```
len(pop); pop.bounds(); pop.radius_stats() # {"min", "max", "mean"}
```

3. Génération

3.1 Cœur : `NumpyGranulo`

`Project.deposit(config)` appelle `NumpyGranulo().deposit(config)` (rejet par lots vectorisés, `seed` reproductible) puis insère la population. Conteneurs gérés :

Dim.	container_type	container_params
2D	Box2D	lx, ly — domaine $[0, lx] \times [0, ly]$ (comme <code>depositInBox2D</code>)
2D	Disk2D, Drum2D	r
2D	Couette2D	rint, rext
3D	Box3D	lx, ly, lz
3D	Sphere3D	r
3D	Cylinder3D	r (ou R), lz

Le résultat (`GranuloResult`) donne `nb_placed`, `nb_requested`, `fill_ratio`. Le nombre placé peut être inférieur au nombre demandé.

3.2 Mode « distribution seule »

`GranuloConfig(create_avatars=False)` : tire seulement `nb_particles` rayons dans `[radius_min, radius_max]`, les stocke dans `config.radii` et ajoute le config à `project.granulo`. Aucune population, aucun avatar.

3.3 Dépôt `pymgc90` (moteur)

`lmgc90_engine.granulo.deposit_population(config)` appelle `pre.depositInBox2D/3D`, `depositInDrum2D/Disk2D`, `depositInCouette2D`, `depositInSphere3D`, `depositInCylinder3D` et renvoie une `ParticlePopulation`. Le contrôleur choisit :

```
contrôleur.deposit(config)
■ create_avatars=False → Project.deposit (rayons seuls)
```

```

■■ pylmgc dispo → session.run_granulo (repli sur NumpyGranulo en cas d'échec)
■■ sinon → Project.deposit (NumpyGranulo)

```

L'onglet **Granulo** expose aussi une case explicite « dépôt pylmgc (engine) » (`run_granulo_pylmgc`).

3.4 Scènes et ForLoop

- Les scènes d'exemple déclarent un `GranuloConfig` dans `project.granulo` ; `ProjectController.apply_scene` appelle ensuite `deposit` pour chaque config sans `population_id`.
- `ForLoop` cible `granulo` → une population par itération ; cible `granulo_dist` → distributions de rayons seules. La cible `avatar` d'un `ForLoop` reste **AoS** (pas de chemin SoA pour les boucles d'avatars).

4. Persistance à deux niveaux

4.1 Sidecar .npz

Fichier	Contenu
<code>projet.lmgc90</code> (JSON)	<code>particle_populations</code> : métadonnées via <code>to_meta_dict()</code> (sans tableaux, avec <code>n_particles</code>), <code>populations_groups</code> , <code>granulo_generations</code> , clé <code>particle_populations_sidecar</code>
<code>projet.populations.npz</code>	"<population_id>_centers" et "<population_id>_radii" (compressé)

```

from lmgc90_core.io import sidecar_path_for, save_populations_sidecar, load_populations_sidecar

sidecar_path_for(Path("p.lmgc90")) # → p.populations.npz
save_populations_sidecar(populations, npz_path)
load_populations_sidecar(npz_path) # {pop_id: (centers, radii)}

```

Règles :

- aucune population → aucun `.npz` écrit ; un ancien sidecar est **supprimé** ;
- sidecar absent → `load_populations_sidecar` renvoie `{}`.

4.2 Rechargement

`io.load_project` fusionne méta JSON + tableaux ; seules les populations dont les tableaux sont trouvés sont restaurées (`if pid in arrays`). Une population dont le tableau manque est **ignorée silencieusement** (il n'y a plus de `load_warnings`).

4.3 Formes de sérialisation

Méthodes	Usage
<code>to_meta_dict()</code> / <code>from_meta_and_arrays(meta, centers, radii)</code>	production (JSON + npz)
<code>to_dict()</code> / <code>from_dict(d)</code>	autonome (tableaux en listes) — tests, duplication

5. Historique et undo

`AddPopulation(population, config)` est la Command d'ajout (une commande quelle que soit la taille), `revert` appelle `_drop_population` (retire population, `GranuloConfig` associé et entrées de `population_groups`). `Project.add(ParticlePopulation)` passe par cette commande. Voir dev.md §12 pour la couverture undo de `Project.deposit` et `controller.remove_population`.

6. Consommateurs de populations

Consommateur	Comportement
<code>pre_script.emit_pre</code>	par population : <code>np.array</code> des centres/rayons, puis boucle <code>for ... pre.rigidXxx(...) + bodies.addAvatar</code> . Fonctions émises : disque, sphère, disque discret, cluster, cylindre (les autres types retombent sur <code>rigidDisk</code>).
<code>engine.materialize</code>	<code>build_population_bodies</code> : une boucle <code>pre.<fn></code> par particule ; résultat dans <code>scene.bodies_by_population_id[pop_id]</code> (et exposé par <code>session.population_map</code>). Les populations sont ajoutées après les avatars AoS dans le conteneur <code>bodies</code> .
<code>viewer_3d</code>	<code>_expand_renderables</code> appelle <code>as_avatar_view(i)</code> pour chaque particule (O(N) objets temporaires), puis fusionne par lots (type + couleur) disque/sphère. Gros volumes : prévoir la lenteur.
<code>scene_geometry</code> (schéma 2D)	sous-échantillonnage à <code>max_population_preview=5000</code> particules.
<code>visibility_tab / dof_tab</code>	utilisent <code>pop.color</code> et <code>pop.avatar_type</code> pour suggérer couleurs/contacteurs ; les DOF ciblent un avatar, un groupe d'avatars ou une couleur (pas une particule).
<code>safe_eval</code>	<code>avatar[...]</code> et <code>group[...]</code> n'exposent que <code>project.avatars / avatar_groups</code> .
<code>compute_script</code>	agnostique : une fois la DATBOX écrite, une particule SoA est un corps <code>RBDY2/3</code> comme les autres. Les groupes d'avatars ne résolvent que les AoS.

7. Comparaison AoS / SoA

Critère	AoS — Avatar	SoA — ParticlePopulation
Stockage	un objet par corps	2 tableaux contigus + méta
Identité	<code>avatar_id</code> stocké	<code>population_id</code> stocké ; id de particule dérivé
Création	<code>Project.add</code> × N (une commande chacun) ; boucles/maçonnerie	<code>Project.deposit</code> — une population
Édition individuelle	native (<code>AvatarTab</code>)	non (population = bloc)
Types	tous	<code>POPULATION_TYPES</code>
Sérialisation	JSON inline (avatars <code>manual</code>)	méta JSON + sidecar <code>.npz</code>
<code>pre.py</code>	une ligne par avatar	une boucle par population
Undo	une commande par avatar	<code>AddPopulation</code> (une commande)
<code>safe_eval</code>	oui	non

Règle de choix : grand nombre de particules strictement homogènes, sans besoin d'édition individuelle → SoA ; sinon AoS.

8. Pièges et conventions

1. **Homogénéité** : une population = un type, un matériau, un modèle, une couleur.
2. **Pas de matérialisation en masse** (`as_avatar_view` en boucle) hors viewer.
3. **Sidecar synchronisé** : le `.npz` est écrit avec le `.lmgc90` ; à copier/déplacer ensemble.
4. **Régénération** : redéposer crée une **nouvelle** population (nouvel `population_id`) ; les anciens ids `pop_xxx:i` deviennent invalides.
5. **Maçonnerie** : ce n'est pas du SoA — les briques restent des `Avatar` individuels.
6. **Nouveau type SoA** : ajouter le type à `POPULATION_TYPES`, valider ses `extra_params` (`_validate_extra`), compléter `build_population_bodies` (`fn_map`) et `pre_script._emit_population`.

9. Checklist contributeur

- [] Création uniquement via `ParticlePopulation.create`
- [] Insertion via `Project.add(pop)` / `Project.deposit` (commande annulable)
- [] `population_groups` mis à jour (fait par `_insert_population`)
- [] Sidecar : rien à faire si l'on passe par `Project.save`
- [] `pre_script`, `avatar_factory`, `viewer_3d` gèrent le nouveau type
- [] Tests : `create`, `to_dict/from_dict`, `to_meta_dict` + aller-retour sidecar, dépôt déterministe (`seed`)

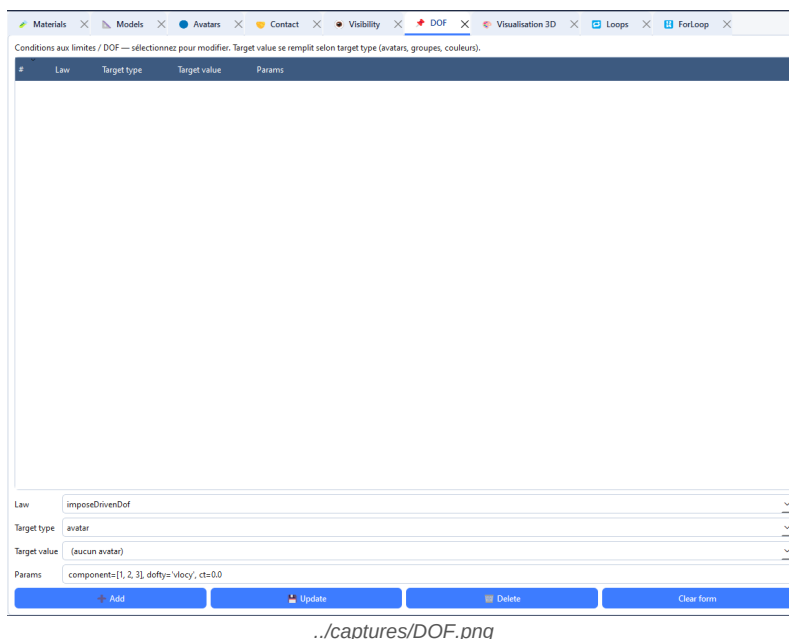
10. Fichiers concernés

Fichier	Rôle
<code>lmgc90_core/population.py</code>	dataclass SoA, validation, vues, sérialisation
<code>lmgc90_core/io.py</code>	sidecar <code>.populations.npz</code> , save/load projet
<code>lmgc90_core/generate.py</code>	<code>NumpyGranulo</code> , <code>GranuloResult</code>
<code>lmgc90_core/project.py</code> , <code>commands.py</code>	<code>deposit</code> , <code>AddPopulation</code> , <code>_insert/_drop_population</code>
<code>lmgc90_core/pre_script.py</code>	<code>_emit_population</code>
<code>lmgc90_engine/granulo.py</code>	dépôt <code>pylmgc90</code> → population
<code>lmgc90_engine/avatar_factory.py</code> , <code>materialize.py</code>	corps <code>pylmgc90</code> par particule
<code>lmgc90_gui/controller/project_controller.py</code>	<code>deposit</code> , <code>run_granulo_pylmgc</code> , <code>remove_population</code>
<code>lmgc90_gui/views/tabs/granulo_tab.py</code>	formulaire de dépôt
<code>lmgc90_gui/views/viewer_3d.py</code>	rendu (expansion $O(N)$)
<code>lmgc90_gui/utils/scene_geometry.py</code>	aperçu 2D sous-échantillonné

Conditions aux limites (DOF)

Conditions aux limites et opérations DOF

L'onglet **DOF** enregistre des opérations cinématiques qui seront appliquées aux corps ciblés lors du calcul. Une opération est définie par une loi, une cible et une chaîne de paramètres.



Ajouter une opération

1. Créez les avatars concernés. Pour viser plusieurs corps ensemble, créez un groupe dans **Groups**.
2. Ouvrez **Onglets** → **Ouvrir** → **DOF** (**Ctrl+9**).
3. Choisissez **Law**.
4. Choisissez **Target type** : **avatar**, **group** ou **color**.
5. Sélectionnez ou saisissez **Target value**. La liste est réapprovisionnée à partir des avatars, groupes et couleurs du projet ; le champ peut aussi être édité.
6. Renseignez **Params** en adaptant les composantes et valeurs à la dimension du projet.
7. Cliquez sur **Add**. L'opération doit apparaître dans la liste.

Opérations disponibles

Law	Rôle	Exemple de paramètres proposés
<code>imposeDrivenDof</code>	Imposer un degré de liberté piloté	<code>component=[1, 2, 3], dofity='vlocy', ct=0.0</code>
<code>imposeInitValue</code>	Définir une valeur initiale	<code>component=[1, 2, 3], dofity='vlocy', values=[0.0, 0.0, 0.0]</code>
<code>translate</code>	Appliquer une translation géométrique	<code>dx=0.0, dy=0.0, dz=0.0</code>

Law	Rôle	Exemple de paramètres proposés
rotate	Appliquer une rotation autour d'un axe	description='axis', axis=[0, 0, 1], alpha=0.0

Ces chaînes sont des exemples de syntaxe affichés par l'interface. Utilisez les composantes qui existent pour le corps et sa dimension ; ne fournissez pas une composante Z dans un cas 2D. Les paramètres sont transmis au type d'opération correspondant, donc respectez les noms attendus (component, dofty, ct, values, dx, etc.).

Choisir la cible

- **avatar** : cible un avatar particulier ; choisissez son entrée dans la liste, qui affiche un identifiant abrégé et son type.
- **group** : cible tous les avatars enregistrés sous le nom du groupe.
- **color** : cible les avatars portant ce code couleur. La liste propose les couleurs présentes dans les avatars/populations et quelques codes LMGC90 usuels.

La cible **avatar** utilise l'identifiant stable du corps, et non sa position dans le tableau. Si un avatar a été supprimé puis recréé, sélectionnez le nouvel avatar dans la liste.

Modifier et supprimer

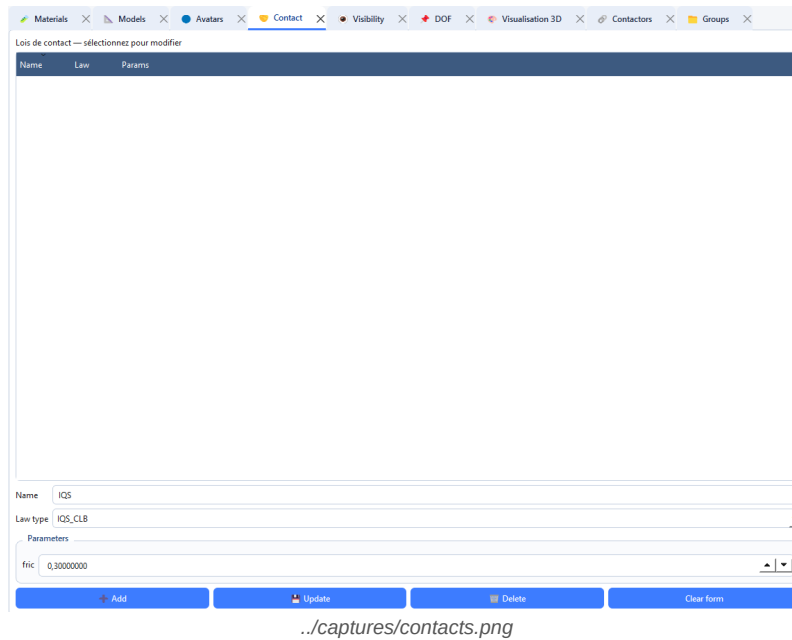
1. Sélectionnez l'opération dans la liste du haut.
2. Modifiez **Law**, **Target type**, **Target value** ou **Params**.
3. Cliquez sur **Update**.
4. Pour la retirer, sélectionnez sa ligne puis cliquez sur **Delete**.
5. Cliquez sur **Clear form** avant de créer une opération indépendante.

Vérifier l'effet visuellement

Ouvrez **Visualisation 3D**, cliquez sur **Rafraîchir la scène**, puis activez **DOF** pour voir les indications correspondant aux opérations configurées. Les symboles sont un contrôle visuel ; ils ne remplacent pas une vérification des paramètres de calcul ni du journal de simulation.

Lois de contact

Une loi de contact décrit la réponse mécanique d'une paire de contacteurs. La table de visibilité associe ensuite les formes et couleurs qui peuvent interagir à une loi nommée.



Créer une loi

1. Ouvrez **Onglets** → **Ouvrir** → **Contact** (Ctrl+7).
2. Saisissez **Name** (maximum cinq caractères) ou gardez la proposition disponible.
3. Sélectionnez **Law type**.
4. Complétez les champs de **Parameters**. Ils changent selon la loi sélectionnée.
5. Cliquez sur **Add** et vérifiez que la loi apparaît dans la liste.

Les lois courantes à contact frottant, telles que **IQS_CLB**, présentent un coefficient **fric**. D'autres lois peuvent présenter des paramètres de cohésion, de restitution, de raideur, de précontrainte ou de rupture ; seuls les paramètres requis par le type courant sont affichés.

Types proposés

Les types exposés par la version actuelle comprennent notamment :

- lois de contact rigide-rigide : **IQS_CLB**, **IQS_CLB_g0**, **IQS_DS_CLB**, **IQS_MOHR_DS_CLB**, **IQS_MAC_CZM**, **RST_CLB** ;
- lois rigide-déformable : **GAP_SGR_CLB**, **GAP_SGR_CLB_g0**, **GAP_MOHR_DS_CLB**, **MAC_CZM**, **MAL_CZM** ;
- liaisons : **ELASTIC_WIRE**, **BRITTLE_ELASTIC_WIRE**, **ELASTIC_ROD**, **VOIGT_ROD** ;
- couplages ou répulsion : **COUPLED_DOF**, **NORMAL_COUPLED_DOF**, **ELASTIC_REPELL_CLB**.

La compatibilité entre loi et paire de contacteurs dépend du type de simulation. Si le choix de paramètres semble inadapté, consultez la documentation LMGC90 de la loi avant le calcul.

Exemples de paramètres fréquemment affichés

Famille	Paramètres possibles dans le formulaire
Coulomb (IQS_CLB)	fric
Variante avec jeu initial (IQS_CLB_g0)	fric, g0
Loi discrète (IQS_DS_CLB)	fric, Rest
Mohr-Coulomb discret	fric, cohes, éventuellement Rest selon la loi
Zone cohésive	w, dn, dt
Fil / barre élastique	stiffness, prestrain ou force_max selon la loi
Barre de Voigt	stiffness, viscosity

Les noms exacts visibles dans **Parameters** sont transmis à pylmgc90. Ne copiez pas le paramètre d'une loi dans une autre si celui-ci n'est pas proposé.

Modifier ou supprimer une loi

1. Cliquez sur sa ligne dans la liste.
2. Vérifiez **Name**, **Law type** et les paramètres rechargés.
3. Modifiez-les puis cliquez sur **Update**.
4. Pour supprimer la loi, sélectionnez-la et cliquez sur **Delete**.
5. Utilisez **Clear form** pour désélectionner l'entrée et préparer une autre loi.

Une loi référencée par une table de visibilité ne peut plus être trouvée par cette table si elle est renommée ou supprimée. Après toute modification, ouvrez **Visibility** et vérifiez les références.

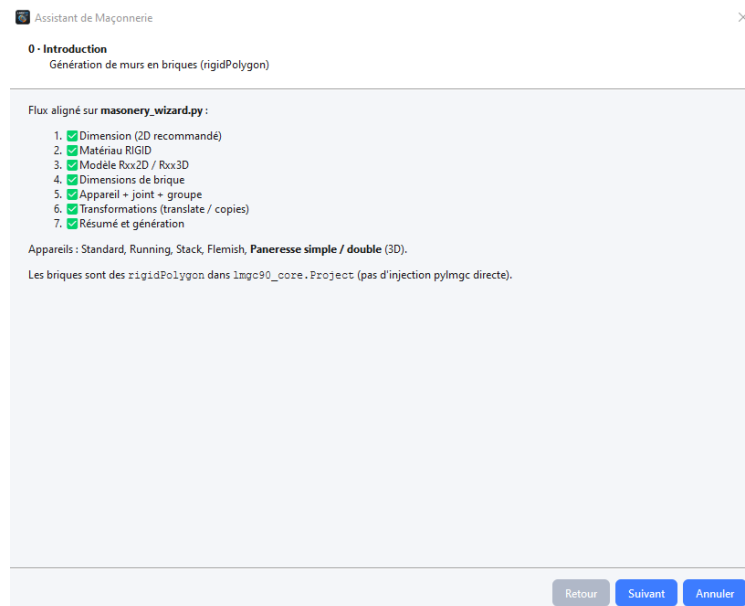
Associer la loi à des contacteurs

1. Vérifiez que les avatars ont leurs formes de contact dans **Contactors** si nécessaire.
2. Créez ou sélectionnez une loi dans **Contact**.
3. Ouvrez **Visibility**.
4. Choisissez les corps, contacteurs et couleurs candidat/antagoniste.
5. Sélectionnez le nom exact de la loi et réglez **Alert**.
6. Cliquez sur **Add**, puis utilisez **Vérifier** pour voir les lois qui ne sont pas référencées ou les références inconnues.

La création d'une loi seule ne l'applique à aucun contact. La table de visibilité est l'étape qui la rattache aux paires concernées.

Assistant de maçonnerie

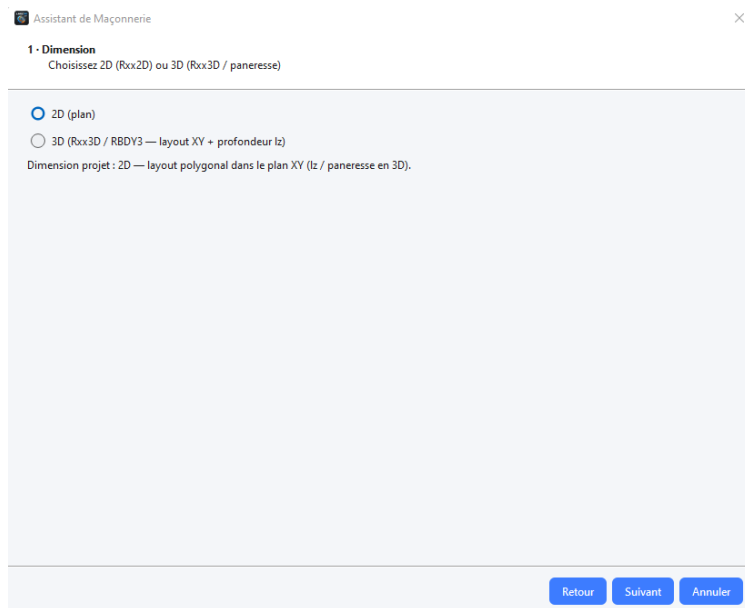
L'assistant **Maçonnerie** construit un assemblage de briques polygonales et peut créer le groupe, la loi de contact et la table de visibilité associés. Ouvrez-le depuis **Tools** → **Assistant Maçonnerie...** (**Ctrl+Shift+M**) ou depuis l'onglet **Masonry**.



../captures/maçon_page1.png

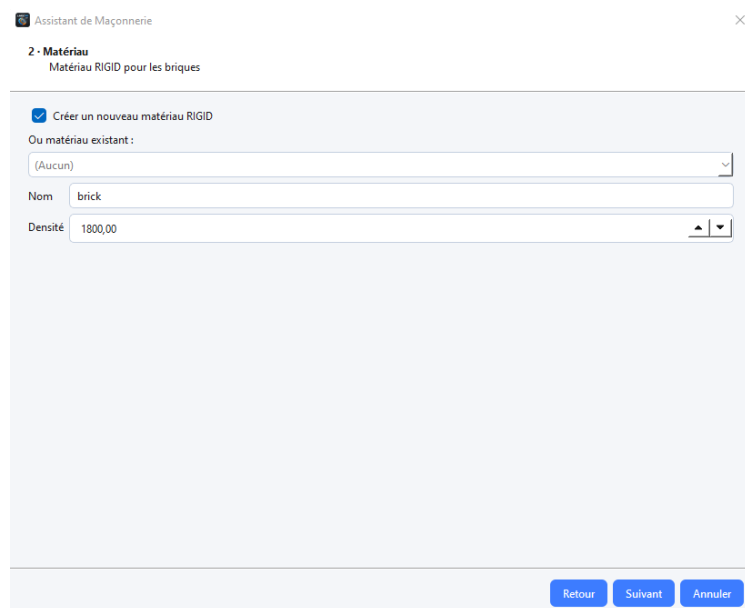
Suivre l'assistant

1. Dans **Introduction**, cliquez sur **Suivant**.
2. **Dimension** : choisissez 2D ou 3D. Les motifs paneresses sont réservés à la 3D.



../captures/maçon_page2.png

3. Matériau : créez un matériau **RIGID** avec sa densité ou sélectionnez un matériau existant.



../captures/maçon_page3.png

4. Modèle : créez un modèle rigide ou réutilisez-en un de dimension compatible (**Rxx2D** en 2D, **Rxx3D** en 3D).

Assistant de Maçonnerie

4 - Dimensions de la brique
lx longueur - ly hauteur - lz profondeur (3D)

Nom brique: stdxx

lx (longueur): 0,2000

ly (hauteur): 0,0650

lz (profondeur / 3D meta): 0,1000

Brique française courante: 0,20 × 0,065 m

Retour Suivant Annuler

../captures/maçon_page4.png

5. Dimensions de la brique : indiquez le nom (cinq caractères maximum), **lx**, **ly** et la profondeur **lz** en 3D.

Assistant de Maçonnerie

5 - Appareil et disposition
Pattern, courses, joints, offsets, options paneress

Appareil: Standard

Courses (rows): 8

Colonnes: 5

Joint (m): 0,0100

Offset X: 0,0000

Offset Y: 0,0000

Offset Z: 0,0000

Couleur: REEDx

Nom de groupe: mason

Loi: IQSxx

Friction: 0,600

Décalage 1/2 brique sur les rangs impairs (appareil courant).

☐ Demi-briques aux extrémités (Standard)

☒ Enregistrer le groupe

☒ Ajouter loi IQS_CLB + see-table (POLYG 2D / POLYR 3D)

Retour Suivant Annuler

../captures/maçon_page5.png

6. **Appareil et disposition** : choisissez le motif, les courses, les colonnes, la largeur du joint, les offsets et la couleur.

7. Choisissez si vous souhaitez enregistrer un **groupe** et définissez son nom.

8. L'option **Ajouter loi IQS_CLB + see-table** est cochée par défaut. Ajustez le nom de loi et la friction, ou décochez-la si vous configurerez les interactions vous-même.

9. **Transformations** : activez **Translation globale** et renseignez **tx**, **ty**, **tz** si nécessaire. Pour dupliquer l'assemblage, activez **Copies additionnelles**, saisissez le nombre de copies et leur décalage **dx**, **dy**, **dz**.

Assistant de Maçonnerie

6 - Transformations

Appliquées après placement des briques

☐ Translation globale

tx: 0,0000

ty: 0,0000

tz: 0,0000

☐ Copies additionnelles

Nombre de copies: 0

dx: 0,0000

dy: 0,0000

dz: 0,0000

Note : la rotation pure n'est pas appliquée côté core dans cette version (métadonnée conservée dans le journal).

Retour Suivant Annuler

../captures/maçon_page6.png

10. Résumé et génération : vérifiez dimensions, matériau, modèle, motif, taille de l'assemblage, groupe, loi et transformations.

11. Cliquez sur Générer. Une confirmation indique le nombre de briques créées.

Assistant de Maçonnerie

7 - Résumé et génération

Vérifiez les paramètres puis cliquez sur Générer

Dimension UI / projet : 2D
 Matériau : brick
 Modèle : rigid
 Brique : stdbx lx=0.2 ly=0.065 lz=0.1
 Appareil : Standard 8x5 joint=0.01
 Origine : (0,0, 0,0)
 Groupe : mason
 Loi IQS : oui

Retour Générer Annuler

../captures/maçon_page7.png

Les boutons **Retour**, **Suivant** et **Annuler** permettent de naviguer dans l'assistant avant la validation finale.

Motifs disponibles

Motif	Disposition générale
Standard	Décalage d'une demi-brique sur les rangs alternés ; option de demi-briques aux extrémités.
Running Bond	Décalage progressif des joints entre rangs.
Stack Bond	Joints alignés verticalement.
Flemish Bond	Alternance de briques en longueur et en boutisse.

Motif	Disposition générale
Paneresse simple (pylmgc90)	Appareillage 3D spécialisé, options dans le groupe paneresse.
Paneresse double (pylmgc90)	Deux feuilles 3D, options dans le groupe paneresse.

Les choix spécifiques au motif s'affichent dans l'étape de disposition : disposition paneresse, première brique, mode de longueur et option sans demi-briques.

Limites et résultat

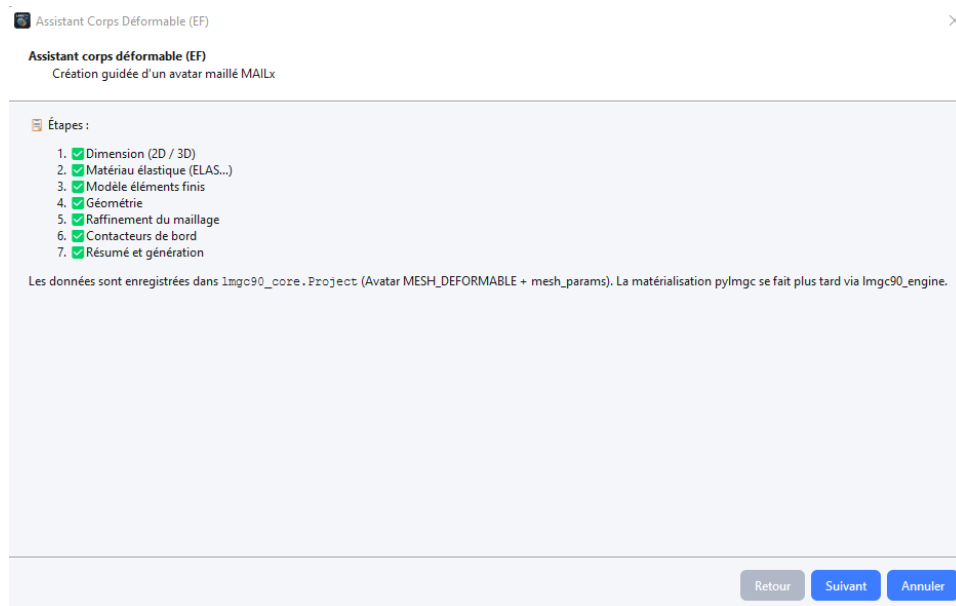
- La rotation pure n'est pas appliquée dans la version actuelle ; l'interface le précise à l'étape **Transformations**.
- Une nouvelle génération ajoute des briques au projet et ne remplace pas les précédentes.
- Si l'assistant crée la loi et la see-table, vérifiez la couleur et le contacteur utilisés dans **Visibility** après génération.
- Les briques sont des avatars du projet. Vous pouvez inspecter le groupe dans **Groups** et actualiser la scène dans **Visualisation 3D**.

Assistant de corps déformable

L'assistant **Mesh déformable** guide la création d'un corps maillé et de ses contacteurs de bord. Il est accessible depuis **Tools** → **Assistant Mesh déformable...** (**Ctrl+Shift+D**) ou l'onglet **Deformable**.

Ouvrir l'assistant

1. Ouvrez **Deformable** depuis **Onglets** → **Ouvrir** et cliquez **Ouvrir l'assistant MeshWizard...**, ou choisissez la commande dans **Tools**.
2. Lisez l'introduction puis cliquez sur **Suivant**. **Retour** revient à la page précédente ; **Annuler** ferme l'assistant.



../captures/defor_page1.png

Parcours des pages

1. **Dimension** : sélectionnez 2D ou 3D conformément au projet. Si des avatars ou modèles existent déjà, leur dimension doit rester cohérente.

Assistant Corps Déformable (EF) ×

Dimension
2D ou 3D — doit rester cohérente avec le projet

☒ 2D (plan)
☐ 3D

Dimension actuelle du projet : 2D

Retour Suivant Annuler

../captures/defor_page2.png

2. Matériau : créez un matériau élastique parmi les types proposés, ou choisissez un matériau existant de même usage. Pour **ELAS**, le choix d'anisotropie (**isotropic** / **orthotropic**) adapte les paramètres affichés : **young**, **nu** et **G** en isotrope ; **E1**, **E2**, **v12** et **G12** en orthotrope 2D ; les constantes de direction 3 supplémentaires (**E3**, **v13**, **v23**, **G13**, **G23**) en orthotrope 3D. Seules les constantes applicables sont enregistrées.

Assistant Corps Déformable (EF) ×

Matériau
Créer un matériau élastique ou en réutiliser un

☒ Créer un nouveau matériau
 Ou choisir un matériau existant :

(Aucun matériau élastique)

Nom ELAS1

Type ELAS

Densité 2500,00

Propriétés

elas standard
 anisotropy isotropic
 young 70000000000,0000
 nu 0,300000
 G 80769230000,0000

Retour Suivant Annuler

../captures/defor_page3.png

3. Modèle EF : créez un modèle ou réutilisez un modèle compatible en dimension. Si vous en créez un, choisissez physique, élément et options proposées (anisotropie, cinématique, formulation et stockage de masse).

Assistant Corps Déformable (EF)

Modèle EF
Élément, physique et options numériques

☒ Créer un nouveau modèle

Ou choisir un modèle existant :
(Aucun modèle 2D)

Nom: femxx

Physique: MECAx

Élément: T3xxx

Anisotropy: Triangle linéaire à 3 noeuds

Kinematic: iso_

Formulation: UpdtL

Mass storage: lump_

Retour Suivant Annuler

../captures/defor_page4.png

4. Géométrie : choisissez une forme disponible :

- en 2D : **Rectangle**, **Disque**, **Fichier externe** ;
- en 3D : **Boîte (H8)**, **Sphère**, **Cylindre**, **Fichier externe**.

Assistant Corps Déformable (EF)

Géométrie
Forme du domaine à mailler

Type: Rectangle

Centre X: 0,000000

Centre Y: 0,000000

Centre Z: 0,000000

lx: 1,000000

ly: 0,400000

lz: 1,000000

Rayon: 0,50

Hauteur: 1,00

Fichier (.msh / .brep / .step...): Parcourir...

Couleur: CYANx

Retour Suivant Annuler

../captures/defor_page5.png

5. Renseignez le centre et les dimensions, rayon ou hauteur demandés par la géométrie.

6. Pour **Fichier externe**, cliquez **Parcourir...** puis choisissez un format affiché dans le sélecteur, tel que **.msh**, **.vtk**, **.brep**, **.step**, **.iges** ou **.geo**.

7. Raffinement : choisissez le type de maillage proposé puis réglez les subdivisions actives : **nx**, **ny**, **nz**, **nr**, **ntheta** ou **nphi**. Des valeurs plus élevées donnent davantage d'éléments et peuvent accroître le coût de calcul.

Assistant Corps Déformable (EF) ×

Raffinement
Subdivision du maillage

mesh_type (2D)	2T3	▼
nx	6	▲ ▼
ny	3	▲ ▼
nz	3	▲ ▼
nr (radial)	4	▲ ▼
ntheta	16	▲ ▼
nphi	12	▲ ▼

Retour Suivant Annuler

../captures/defor_page6.png

8. Contacteurs de bord : conservez **Ajouter un contacteur de bord** si le corps doit interagir. Choisissez sa forme compatible avec la dimension et sa couleur. **by group** est facultatif.

Assistant Corps Déformable (EF) ×

Contacteurs de bord
Surfaces de contact sur le maillage (CLxxx / CSpxx...)

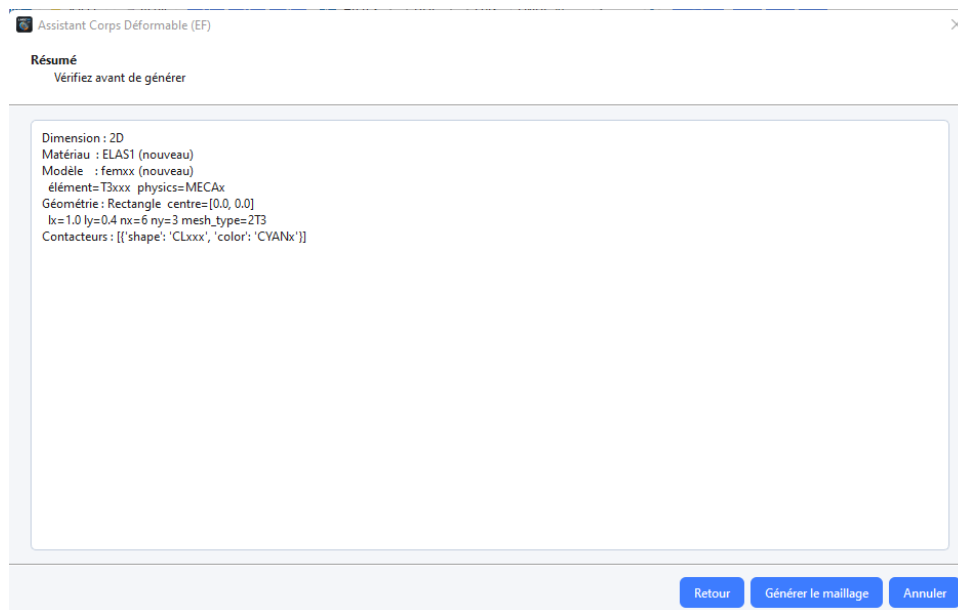
☒ Ajouter un contacteur de bord

Shape	CLxxx	▼
Color	CYANx	
by group	optionnel — groupe de noeuds/faces	

Retour Suivant Annuler

../captures/defor_page7.png

9. Résumé : vérifiez la dimension, le matériau, le modèle, la géométrie, le raffinement et les contacteurs.



../captures/defor_page8.png

10. Cliquez sur **Générer le maillage**. L'assistant confirme la création ou affiche l'erreur à corriger.

Formes et maillage effectivement produits

Le rectangle 2D utilise la construction structurée disponible dans l'application. Pour les autres géométries et les fichiers externes, l'assistant enregistre l'intention de maillage et ses paramètres dans le projet ; la génération/matérialisation complète dépend de l'export et des outils LMGC90 présents dans l'environnement.

Le bouton **Parcourir...** enregistre un chemin : ce n'est pas un aperçu du maillage importé. Après la génération de l'avatar, contrôlez le résultat dans le projet et lors de la préparation du calcul.

Après l'assistant

- L'avatar déformable apparaît dans **Avatars**.
- Les contacteurs peuvent être vérifiés ou modifiés dans **Contactors**.
- La loi et les paires d'interaction se configurent dans **Contact** et **Visibility**.
- Pour appliquer des conditions, utilisez **DOF**.

Voir également Contacteurs, Tables de visibilité et Conditions aux limites.

Préparer et lancer un calcul

L'onglet **Compute** configure le script de calcul `command.py`, prépare le répertoire de travail et peut lancer le processus chipy. La disponibilité de pylmgc90 dans l'environnement qui exécute LMGC90_GUI détermine les fonctions natives accessibles.

Ouvrir l'onglet Compute

- Choisissez **Onglets** → **Ouvrir** → **Compute** lorsque cette entrée est disponible dans votre construction de l'application.
- Le raccourci **F5** depuis la fenêtre principale ouvre également l'onglet et déclenche l'action de calcul.

L'onglet affiche un résumé de la dimension, du nombre d'avatars et de particules, des lois, tables de visibilité, DOF et corps déformables détectés. Vérifiez qu'il correspond au projet préparé.

Régler les paramètres de base

1. Dans **Paramètres temporels**, définissez **Pas de temps (dt)**, **Nombre d'itérations** et **Theta intégrateur**.
2. Dans **Solveur de contact**, réglez **Tolérance**, **Relaxation**, **Norme**, **Itérations GS1**, **Itérations GS2** et **Type de solveur**.
3. Dans **Sorties**, indiquez les fréquences **WriteOut** et **Display**.
4. Activez ou désactivez **Disable chipy log messages** selon que vous souhaitez réduire la sortie standard.
5. Activez **ReadDatbox(deformable=True)** si la scène comporte des corps déformables. Le résumé peut également détecter un maillage.

Les valeurs initiales de l'interface sont des paramètres de départ, pas une garantie de stabilité ou de convergence. Adaptez le pas de temps, le solveur et les fréquences à l'échelle du problème.

Configurer les routines chipy

1. Cliquez sur **Configurer les routines chipy...**
2. Dans **Modele**, vérifiez l'hypothèse, la physique et les paramètres de modèle proposés.
3. Dans **Routines**, activez les familles de corps et détecteurs nécessaires aux paires de contact du projet. Pour des corps déformables, activez également les routines FEM et les contacteurs mixtes pertinents.
4. Dans **Extraction**, choisissez visualisation, visibilité des avatars, vecteurs d'état, forces et énergies requis.
5. Dans **Pilotage**, configurez au besoin le redémarrage, le critère d'arrêt et les pas de temps multiples.
6. Réglez les onglets **Inspect. 2D**, **Inspect. 3D** et **Inspect. Interact.** si des fonctions d'inspection sont nécessaires.
7. Utilisez **Aperçu du script command.py** pour examiner le script généré.
8. Validez avec **OK** ou abandonnez avec **Cancel**. **Restore Defaults** rétablit les valeurs par défaut du dialogue.

Les détecteurs cochés doivent couvrir les types de contacteurs effectivement présents et les tables de visibilité du projet. Une table ne suffit pas si le détecteur correspondant n'est pas activé.

Choisir le répertoire et préparer les fichiers

1. Dans **Répertoire de calcul**, saisissez le chemin de travail ou cliquez sur **Parcourir...**.
2. Cliquez sur ■ **Sauver** → **préférences** si vous souhaitez réutiliser ces réglages ultérieurement.
3. Cliquez sur ■ **Préparer DATBOX / scripts**.
4. L'application écrit `pre.py` et `command.py` dans le répertoire sélectionné.
5. La création de **DATBOX** dépend des préférences d'écriture automatique et de la disponibilité de pylmgc90.
6. Vérifiez le message de confirmation ainsi que le journal de préparation.

Le menu **Computation** → **Generate DATBOX / scripts...** (**Ctrl+F5**) offre également une action de préparation/export. Pour exporter uniquement les scripts ou l'ensemble depuis des boîtes de dialogue de sauvegarde, utilisez **Tools** → **Generate pre.py...**, **Generate command.py...** ou **Export all...**

Lancer et arrêter le calcul

1. Vérifiez le chemin du répertoire de travail.
2. Cliquez sur ■ **Lancer le calcul (F5)** ou utilisez **F5**.
3. Si `command.py` est absent, acceptez sa préparation si vous souhaitez poursuivre.
4. Confirmez le répertoire et la commande si la confirmation de lancement est activée dans les préférences.
5. Suivez les lignes du journal dans l'onglet Compute.
6. Cliquez sur ■ **Stop** pour demander l'arrêt du processus.
7. À la fin, vérifiez le statut et le code de retour (**rc**).

Le processus tourne séparément afin de laisser l'interface réactive. Un lancement exige un interpréteur Python et des dépendances LMGC90 fonctionnels dans l'environnement sélectionné.

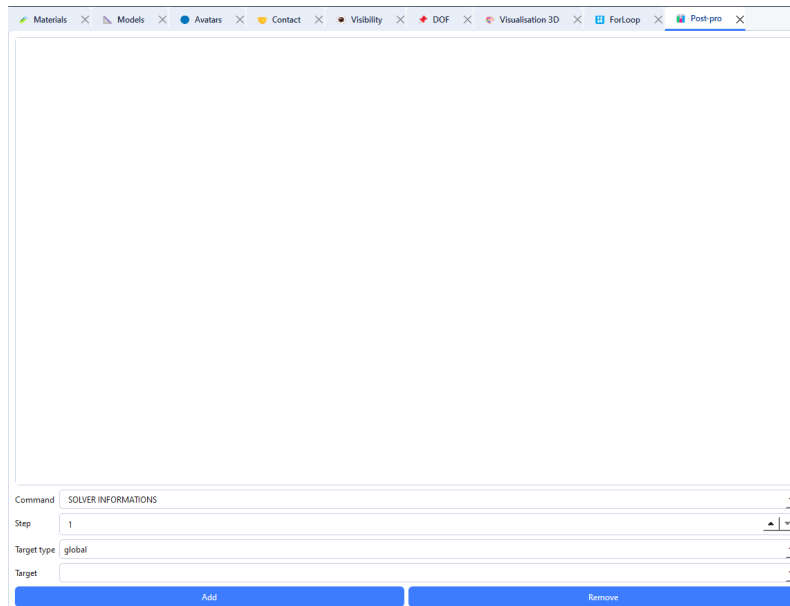
Préférences et diagnostic

- ■ **Charger préférences** recharge les paramètres enregistrés.
- ■ **Sauver** → **préférences** mémorise les paramètres de calcul et le répertoire de travail.
- Le journal de calcul contient les sorties du processus.
- **Computation** → **Application journal (F7)** fournit le journal interne de l'application.

La page Post-traitement décrit les commandes d'extraction ; la page Visualisation décrit l'inspection de la scène avant calcul. L'application actuelle ne fournit pas un écran complet de parcours des fichiers de résultats calculés.

Configurer le post-traitement

L'onglet **Post-pro** ajoute des commandes d'extraction au calcul. Il configure ce que le calcul doit écrire ; il ne sert pas à ouvrir ou analyser les fichiers de résultats après exécution.



../captures/postpro.png

Ajouter une commande

1. Ouvrez **Onglets** → **Ouvrir** → **Post-pro**.
2. Choisissez une valeur de **Command** ou saisissez le nom d'une commande acceptée par votre installation LMG90.
3. Réglez **Step**, la fréquence d'écriture.
4. Choisissez **Target type** : **global**, **avatar** ou **group**.
5. Pour une cible **avatar**, sélectionnez l'avatar ; pour **group**, choisissez le groupe. Pour **global**, aucune cible n'est nécessaire.
6. Cliquez sur **Add**.
7. Vérifiez la commande dans la liste : nom, fréquence et cible.

Les noms préremplis comprennent notamment **SOLVER INFORMATION**, **BODY TRACKING**, **TORQUE EVOLUTION**, **KINETIC ENERGY**, **COORDINATE** et **VAN_DER_WAALS**. Les commandes doivent être compatibles avec les données et le calcul configurés.

Retirer une commande

1. Sélectionnez la commande dans la liste.
2. Cliquez sur **Remove**.

L'onglet ne propose pas d'édition en place : retirez une commande mal définie puis recréez-la avec les bons paramètres.

Extractions avancées des routines chipy

Pour régler les sorties avancées, ouvrez **Compute**, puis **Configurer les routines chipy....** Le dialogue comprend notamment les onglets **Extraction**, **Inspect. 2D**, **Inspect. 3D** et **Inspect. Interact..** Ils configurent les vecteurs d'état, forces, énergie, champs FEM, visibilité et inspecteurs selon le type de corps.

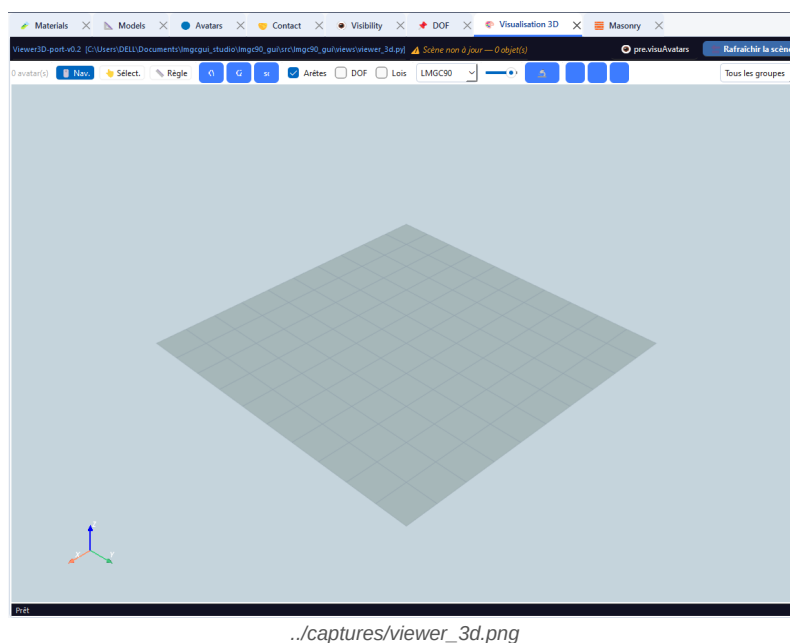
Consulter les journaux

- Le journal de l'onglet **Compute** affiche la sortie du processus de calcul.
- **Computation** → **Application journal** (F7) affiche les erreurs et événements internes de LMGC90_GUI.

Ces journaux ne sont pas un visualiseur des résultats calculés. Pour inspecter la scène de prétraitement, utilisez Visualisation.

Visualisation de la scène

LMGC90_GUI propose deux voies pour inspecter le modèle : le **visualiseur 3D intégré** (PyVista) et le visualiseur natif de pylmgc90 lancé avec **pre.visuAvatars**. Ces outils montrent la scène préparée ; ils ne remplacent pas un outil d'analyse des résultats de calcul.



../captures/viewer_3d.png

Ouvrir et actualiser le visualiseur intégré

1. Ouvrez **Onglets** → **Ouvrir** → **Visualisation 3D**.
2. Vérifiez que le projet contient des avatars ou une population.
3. Cliquez sur **Rafraîchir la scène**. Le transfert vers le viewer est manuel : après une modification du projet, relancez cette action.
4. L'indicateur de la barre du viewer signale si la scène affichée doit être actualisée et indique le nombre d'objets envoyé.

Le viewer intégré requiert les modules `pyvista` et `pyvistaqt`. Si l'un manque, le panneau affiche un message d'indisponibilité au lieu de la scène.

Naviguer dans la scène

- **Nav.** : mode caméra. Faites glisser le bouton gauche pour orbiter, utilisez la molette pour zoomer et le bouton droit pour déplacer la caméra.
- **XY, XZ, Iso** : vues prédéfinies. La vue **XY** convient généralement aux projets 2D. La vue **YZ** n'est pas proposée dans cette version.
- **■ Réinitialiser la caméra** : rétablit le cadrage initial.
- **Arêtes** : affiche ou masque les arêtes des géométries.

- **Curseur d'opacité** : rend les avatars plus ou moins transparents.
- **■ Effacer** : retire les objets de la vue courante. Cliquez à nouveau sur **Rafraîchir la scène** pour repeupler le viewer depuis le projet.

Sélectionner et mesurer

1. Cliquez sur **Sélect.**.
2. Cliquez sur un avatar dans la scène. Il est mis en évidence et ses informations sont indiquées dans la barre d'état.
3. Pour mesurer une distance, cliquez sur **Règle**.
4. Cliquez une première fois sur un point de la scène (A), puis sur le second point (B). La distance est affichée par le viewer.
5. Revenez au mode **Nav.** pour reprendre la navigation.

La sélection porte sur les objets représentés par le visualiseur. En cas de scène vide ou d'objets trop petits à l'écran, réinitialisez la caméra ou zoomez avant de sélectionner.

Lire les couleurs et annotations

1. Choisissez le mode dans la liste de couleurs : **LMGC90**, **Par type**, **Par matériau** ou **Par origine**.
2. Activez **DOF** pour afficher les indices visuels des opérations cinématiques configurées dans l'onglet DOF.
3. Activez **Lois** pour visualiser les liens issus des tables de visibilité (couleur candidate vers couleur antagoniste).
4. Ouvrez le filtre de groupe et choisissez un groupe pour isoler ses avatars. **Tous les groupes** réaffiche toute la scène.

Les couleurs par matériau et par origine servent à l'inspection visuelle ; elles ne changent ni les couleurs enregistrées dans les avatars ni les règles de visibilité.

Exporter une image

1. Cadrez la scène avec la caméra et choisissez les filtres, couleurs et annotations à inclure.
2. Cliquez sur **■ Exporter en PNG**.
3. Choisissez l'emplacement et le nom de l'image dans la boîte de dialogue.

Lancer la visualisation native pylmgc90

1. Dans l'onglet **Visualisation 3D**, cliquez sur **pre.visuAvatars**.
2. Si pylmgc90 est disponible, l'application matérialise la scène puis ouvre le visualiseur natif.
3. Si le bouton indique que **pylmgc90** est indisponible, vérifiez que l'environnement Python utilisé pour lancer LMG90_GUI contient bien cette bibliothèque.

Ce mode utilise les objets pylmgc90 réels et est distinct du rendu paramétrique PyVista. Il est indisponible sans pylmgc90 et peut désactiver la sélection individuelle du viewer intégré pendant son utilisation.

Problèmes courants

Symptôme	Vérification
La scène ne reflète pas la dernière modification	Cliquer sur Rafraîchir la scène .
Le viewer affiche « pyvista + pyvistaqt absents »	Installer ces dépendances dans l'environnement de l'application puis relancer celle-ci.

Symptôme	Vérification
<code>pre.visuAvatars</code> indique pylmgc90 indisponible	Vérifier l'interpréteur/env utilisé par <code>lmgc90-gui</code> , et pas seulement un autre terminal.
Aucun avatar n'apparaît après actualisation	Vérifier que le projet contient des avatars/populations, puis réinitialiser la caméra.

LMGC90_GUI Studio — Architecture & Guide du Contributeur

Monorepo lmgc90gui_studio : lmgc90_core 0.1.x · lmgc90_engine 0.1.x · lmgc90_gui 0.1.x (fenêtre « v0.6.x »). Cette révision remplace l'ancien guide « MVC monolithique » (src/core, ProjectState, _pylmgc_bodies, mixins du contrôleur). Les correspondances ancien → nouveau sont données au §1.2.

Table des matières

1. Vue d'ensemble
2. Structure des fichiers
3. lmgc90_core — le modèle scientifique
4. lmgc90_engine — le pont pylmgc90
5. lmgc90_gui — le client PyQt6
6. Flux de données
7. Systèmes clés
8. Cycle de vie d'un projet
9. Conventions et patterns
10. Guide de contribution
11. Tests
12. Points de vigilance connus

1. Vue d'ensemble

1.1 Trois paquets, une seule direction de dépendance

```

■ lmgc90_gui (PyQt6) ■
■ vues · onglets · assistants · ProjectController (façade) ■
■ dépend de
■ lmgc90_engine (pylmgc90 optionnel, pas de Qt) ■
■ EngineSession · materialize · DATBOX · dépôt pylmgc ■
■ dépend de
■ lmgc90_core (numpy seul, pas de Qt, pas de pylmgc90) ■
■ Project · entités · historique · validation · scripts ■

```

Paquet	Rôle	Dépendances
<code>lmgc90_core</code>	Source de vérité : <code>Project</code> , entités, <code>CommandHistory</code> (undo/redo), validation, générateurs numpy, sérialisation <code>.lmgc90</code> + sidecar <code>.npz</code> , émission <code>pre.py</code> / <code>command.py</code> / <code>run.sbatch</code> , scènes d'exemple.	<code>numpy</code>
<code>lmgc90_engine</code>	<code>Project</code> → vrais objets <code>pylmgc90.pre</code> , écriture DATBOX, <code>pre.visuAvatars</code> , dépôt granulo <code>pylmgc</code> .	<code>lmgc90_core</code> , <code>numpy</code> , <code>pylmgc90</code> (<i>optionnel</i>)
<code>lmgc90_gui</code>	Client Qt : onglets CRUD, assistants, viewer 3D PyVista, journal, calcul.	<code>lmgc90_core</code> , <code>lmgc90_engine</code> , <code>PyQt6</code> ; <code>pyvista/pyvistaqt</code> (<i>optionnels</i>)

1.2 Règles d'architecture (non négociables)

- **Pureté** : `lmgc90_core` n'importe jamais `pylmgc90`, `PyQt6`, `pyvista`, `gms`, `vtk` (garanti par `lmgc90_core/tests/test_purity.py`). Le moteur n'importe `pylmgc90` que dans des fonctions (`test_engine_purity.py`).
- **Identité** = `avatar_id` / `population_id`, jamais un indice de liste.
- **Le contrôleur ne possède aucun objet pylmgc90**. Les objets « live » vivent dans `EngineSession` après `materialize()`.
- **Une seule source de rafraîchissement** : le signal `state_changed`.
- **Pas de logique métier dans les onglets** : ils appellent `controller.*`.
- **AoS vs SoA** : `Avatar` = édition fine ; `ParticlePopulation` = génération de masse (SoA est le comportement **par défaut** du dépôt, plus une case à cocher).
- `pre.py` **généré** = **artefact de reproductibilité** (plus que le JSON).

1.3 Correspondance ancienne → nouvelle architecture

Ancien (MVC monolithique)	Nouveau
<code>src/core/models.py</code> (<code>ProjectState</code>)	<code>lmgc90_core/entities.py</code> + <code>lmgc90_core/project.py</code> (<code>Project</code>)
<code>src/core/validators.py</code>	<code>lmgc90_core/validate.py</code> (fonctions <code>validate_*</code>)
<code>src/core/generators.py</code>	<code>lmgc90_core/generate.py</code> (<code>loop_positions</code> , <code>expand_loop</code> , <code>expand_for_loop</code> , <code>NumpyGranulo</code>)
<code>src/core/particle_population*.py</code>	<code>lmgc90_core/population.py</code> + <code>lmgc90_core/io.py</code> (sidecar)
<code>src/core/serializers.py</code>	<code>lmgc90_core/io.py</code>
<code>src/core/pylmgc_bridge.py</code> (<code>LMGC90Bridge</code>)	<code>lmgc90_engine/materialize.py</code> + <code>avatar_factory.py</code>
<code>src/utils/script_generator.py</code>	<code>lmgc90_core/pre_script.py</code> (<code>emit_pre</code>)
<code>src/utils/compute_script_generator.py</code>	<code>lmgc90_core/compute_script.py</code> (shim déprécié côté GUI)
<code>src/utils/fast_granulo_engin.py</code>	<code>NumpyGranulo</code> (core) — il n'y a plus de moteur « fast » séparé
<code>controllers/<i>_mixin.py, _pylmgc_</i></code>	<code>ProjectController</code> mince + <code>EngineSession</code>
<code>controller.state</code>	<code>controller.project</code>
<code>pre.* pylmgc90</code> dans la GUI	<code>lmgc90_core.pre</code> (fabriques pures aux mêmes kwargs)
<code>avatar[i]</code> positionnel	toujours exposé en <code>safe_eval</code> , mais l'identité réelle est <code>avatar_id</code>

Ancien (MVC monolithique)	Nouveau
Case à cocher « SoA »	Project.deposit produit toujours une ParticlePopulation
Particle Factory, Convertisseur de script, Templates, Assistants projet/granulo	non portés dans la version actuelle

2. Structure des fichiers

```

lmgc90gui_studio/
■■■ README.md / README_fr.md / README_INSTALL.md
■■■ install.ps1 # core → engine → gui[qt]
■■■ pyproject.toml # méta (ne remplace PAS les 3 paquets)
■■■ docs/fr/ # documentation utilisateur + technique
■
■■■ lmgc90_core/
■   ■■■ pyproject.toml
■   ■■■ examples/ # granular_box.py, uniaxial_compression.py
■   ■■■ tests/ # test_purity, test_project, test_population, ...
■   ■■■ src/lmgc90_core/
■   ■■■ __init__.py # API publique (Project, pre, entités, ...)
■   ■■■ types.py # enums + schémas (matériaux, lois, éléments)
■   ■■■ ids.py # avatar_id / population_id / id de particule
■   ■■■ errors.py # LMGC90Error, ValidationError, ...
■   ■■■ entities.py # Material, Model, Avatar, ContactLaw, ...
■   ■■■ population.py # ParticlePopulation (SoA)
■   ■■■ project.py # Project (agrégat + API add/undo/save)
■   ■■■ commands.py # Command, CommandHistory
■   ■■■ validate.py # validate_material/model/avatar/...
■   ■■■ pre.py # fabriques « comme pylmgc90.pre »
■   ■■■ generate.py # boucles, ForLoop, NumpyGranulo
■   ■■■ masonry.py # MasonryConfig, expand_masonry
■   ■■■ io.py # .lmgc90 JSON + .populations.npz
■   ■■■ pre_script.py # emit_pre / emit_equivalent
■   ■■■ chipy_script.py # emit_chipy
■   ■■■ compute_script.py # ComputeScriptGenerator (command.py)
■   ■■■ pipeline.py # Pipeline, render_sbatch
■   ■■■ numpy_compat.py # patch np.cross (NumPy 2 / 2D)
■   ■■■ scenes/ # scènes de démonstration pures
■   ■■■ data/meshes/Carre.msh
■
■■■ lmgc90_engine/
■   ■■■ pyproject.toml
■   ■■■ examples/session_demo.py
■   ■■■ tests/
■   ■■■ src/lmgc90_engine/
■   ■■■ session.py # EngineSession
■   ■■■ materialize.py # Project → conteneurs pre + MaterializedScene
■   ■■■ avatar_factory.py # Avatar/Population → corps pre
■   ■■■ granulo.py # dépôt pylmgc90 → ParticlePopulation
■   ■■■ datbox.py # writeDatbox, DATBOX dual (cellule)
■   ■■■ errors.py
■
■■■ lmgc90_gui/
■■■ pyproject.toml # extras [qt] [viz] [dev]
■■■ main.py
■■■ examples/ # headless_demo.py, wave1_scene.py
■■■ tests/
■■■ src/lmgc90_gui/
■■■ app.py # main() + QApplication
■■■ controller/

```



```

■ ■ ■ ■ project_controller.py
■ ■ ■ ■ signals.py # pyqtSignal ou stubs sans Qt
■ ■ ■ ■ views/
■ ■ ■ ■ main_window.py # create_main_window()
■ ■ ■ ■ tree_view.py # Model tree (lecture seule)
■ ■ ■ ■ history_dock.py # pile de commandes + Undo/Redo
■ ■ ■ ■ viewer_3d.py # Viewer3D (PyVista)
■ ■ ■ ■ styles.py
■ ■ ■ ■ widgets/entity_tree.py
■ ■ ■ ■ tabs/ # material, model, avatar, loop, for_loop,
■ # granulo, contact, visibility, dof, postpro,
■ # viewer, groups, contactors, deformable,
■ # masonry, compute (+ base_tab.py)
■ ■ ■ ■ dialogs/ # wizards, préférences, journal, palette, ...
■ ■ ■ ■ workers/ # compute_worker, datbox_worker, materialize_worker
■ ■ ■ ■ examples/ # catalogue ExampleSpec (métadonnées)
■ ■ ■ ■ utils/ # safe_eval, eval_context, preferences,
■ # app_journal, naming, field_forms, scene_geometry
■ ■ ■ ■ resources/

```

3. lmgc90_core — le modèle scientifique

3.1 types.py — vocabulaire LMGC90

Enums (valeurs = noms `pylmgc90`) : `MaterialType`, `AvatarType`, `ContactLawType`, `AvatarOrigin` (`MANUAL`, `LOOP`, `GRANULO`, `FACTORY`), `UnitSystem`.

Tables de référence qui pilotent validation et formulaires GUI :

Constante	Usage
<code>ELEMENTS_BY_PHYSICS[physique][dim]</code>	éléments autorisés (validation + <code>ModelTab</code>)
<code>MATERIAL_PROPERTY_SCHEMA</code> , <code>MATERIAL_PROPERTY_CHOICES</code> , <code>ORTHOTROPIC_FIELDS_2D/3D</code> , <code>ISOTROPIC_FIELDS</code>	champs de <code>MaterialTab</code>
<code>CONTACT_LAW_PROPERTY_SCHEMA</code> , <code>CONTACT_LAW_CATEGORIES</code>	champs de <code>ContactTab</code> , validation des lois
<code>CONTACTORS_RIGID_2D/3D</code> , <code>CONTACTORS_MESH_2D/3D</code>	formes de contacteurs compatibles
<code>RIGID_AVATARS_2D/3D</code> , <code>DEFORMABLE_2D/3D</code>	cohérence avatar ↔ modèle
<code>POPULATION_TYPES</code>	types éligibles au SoA (disque, sphère, disque discret, cluster, cylindre, polygone, polyèdre)
<code>LOOP_ALIASES</code>	noms de boucles (FR/EN) → canoniques

3.2 entities.py — entités sérialisables

Dataclasses sans dépendance Qt/pylmgc : `Material`, `Model`, `Avatar`, `ContactLaw`, `VisibilityRule`, `DOFOperation`, `PostProCommand`, `Loop`, `ForLoop`, `GranuloConfig`. Chacune expose `to_dict()` / `from_dict()`.

```

@dataclass
class Avatar:
    avatar_type: AvatarType
    center: list[float]
    material_name: str
    model_name: str
    color: str = "BLUEx"

```

```

origin: AvatarOrigin = AvatarOrigin.MANUAL
avatar_id: str = field(default_factory=new_avatar_id) # stable
radius: Optional[float] = None
axis: Optional[dict] = None # axe1/axe2/axe3
vertices, nb_vertices, generation_type, is_hollow
wall_params, contactors, mesh_params

```

GranuloConfig = **intention** d'un dépôt (les tableaux vivent dans **ParticlePopulation**). **create_avatars=False** = « distribution seule » (rayons échantillonnés dans **config.radii**, aucune population).

3.3 ids.py

```

new_avatar_id() # uuid4 hex
new_population_id() # "pop_" + uuid4 hex
particle_id(pop_id, i) # "pop_xxx:42"
parse_particle_id("pop_xxx:42") # → ("pop_xxx", 42)

```

3.4 validate.py

Fonctions levant **ValidationError** : **validate_material**, **validate_model**, **validate_avatar**(**avatar**, **model**), **validate_contact_law**, **validate_visibility**(**rule**, **law_names**), plus **compatible_contactors**(**type**, **dim**) et **is_shape_compatible**. Règles clés : noms ≤ 5 caractères, densité > 0, élément valide pour (physique, dimension), avatar rigide ↔ **Rxx2D/Rxx3D**, contacteurs de 5 caractères.

3.5 Project et CommandHistory

```

p = Project(name="box", dimension=2)
p.add(pre.material(name="STEEL", materialType="RIGID", density=7800)) # valide + journalise
p.add(pre.model(name="rigid", physics="MECAx", element="Rxx2D", dimension=2))
av = p.add(pre.rigidDisk(r=0.1, center=[0, 0], model="rigid", material="STEEL"))
p.undo(); p.redo()

```

Champs principaux : **materials**, **models**, **avatars**, **populations**, **laws**, **visibility**, **operations**, **postpro**, **loops**, **for_loops**, **granulo**, **avatar_groups**, **population_groups**, **dynamic_vars**, **history**.

Points d'entrée :

Méthode	Rôle
add(obj, on_conflict=None)	dispatch par type ; valide ; exécute une Command . on_conflict ∈ error (défaut) · skip · merge
group(name, ids) / remove_avatar(id)	groupes et suppression (commandes SetGroup , RemoveAvatar)
deposit(config)	dépôt numpy → une ParticlePopulation ; ou rayons seuls si create_avatars=False
apply_loop, apply_for_loop, apply_masonry	génération d'avatars/entités
undo() / redo() / journal()	historique
save(path) / Project.load(path)	.lmgc90 + sidecar
to_pre_script() / to_chipy_script(params) / sbatch(slurm) / equivalent(obj)	exports
n_bodies, summary()	statistiques

commands.py définit une **Command** (**apply**, **revert**, **describe**, **to_journal**) par action : **AddMaterial**, **AddModel**, **AddAvatar**, **RemoveAvatar**, **AddPopulation**, **AddLaw**, **AddSeeTable**, **AddDOF**, **AddPostPro**, **AddLoop**, **SetGroup**. Granularité voulue : **une population = une commande**, **une boucle = une commande**. **CommandHistory** (limite 200)

séréalise ses piles (`to_dict` / `load_stacks`).

3.6 `pre.py` — fabriques « comme `pymgc90` »

Mêmes noms/kwargs que `pymgc90.pre`, mais retournent des dataclasses pures : `material`, `model`, `rigidDisk`, `rigidSphere`, `rigidJonc`, `rigidPolygon`, `rigidCluster`, `rigidCylinder`, `rigidPlan`, `rigidPolyhedron`, `rigidOvoidPolygon`, `rigidDiscreteDisk`, `smoothWall`, `roughWall`, `finewall`, `granuloRoughWall`, `roughWall3D`, `granuloRoughWall3D`, `emptyAvatar`, `meshed_rectangle`, `tact_behav`, `see_table`.

```
from lmgc90_core import pre
pre.see_table(CorpsCandidat="RBDY2", candidat="DISKx", colorCandidat="BLUEx",
CorpsAntagoniste="RBDY2", antagoniste="JONCx", colorAntagoniste="GRAYx",
behav="IQS01", alert=0.05)
```

3.7 `generate.py`, `masonry.py`

- `loop_positions` / `expand_loop` : cercle, grille (carrée), ligne, spirale ; clone le template (`copy()`, nouvel `avatar_id`, `origin=LOOP`).
- `expand_for_loop` : expressions `expr_x/y/z/radius` évaluées par `_safe_arith` (arithmétique AST restreinte). Les autres cibles de `ForLoop` (`material`, `model`, `dof`, `visibility`, `granulo`, `granulo_dist`) sont gérées dans `Project.apply_for_loop`.
- `NumpyGranulo.deposit(config)` : rejet par lots vectorisés, conteneurs `Box2D`, `Disk2D/Drum2D`, `Couette2D`, `Box3D`, `Sphere3D`, `Cylinder3D`. Déterministe avec `seed`.
- `expand_masonry(MasonryConfig)` : appareils `standard`, `running`, `stack`, `flemish`, `paneresse_simple`, `paneresse_double` ; briques = `rigidPolygon` (2D) ou `rigidPolyhedron` (8 sommets, 3D).

3.8 `population.py` — SoA

Voir Populations de particules.

3.9 `io.py` — persistance

Schéma `.lmgc90 v2` (migration `v1→v2` incluse). Voir §7.3 et la page `particle_population.md` pour le sidecar.

3.10 Scripts

Module	Sortie
<code>pre_script.emit_pre(project)</code>	<code>pre.py</code> auditable (inclut <code>PRE_PY_CROSS_PATCH</code> NumPy 2)
<code>compute_script.ComputeScriptGenerator</code> / <code>generate_command_script</code>	<code>command.py</code> chipy complet (détecteurs, extraction, inspection, restart, critère d'arrêt, multi-pas)
<code>chipy_script.emit_chipy(project, **params)</code>	complète les défauts puis délègue au générateur
<code>pipeline.Pipeline.standard(...)</code> , <code>render_sbatch(...)</code>	pipeline en données + script SLURM

3.11 `scenes/`

`scenes/demos.py` : fonctions `f(project) -> None` (« scène pure », sans `pre`, sans engine) enregistrées dans `SCENE_BUILDERS`. Helpers dans `scenes/_common.py` (`ensure_rigid_2d`, `iqs`, `see_table`, `disk`, `smooth_wall`, `mesh_rect...`). Les dépôts sont déclarés par `project.granulo.append(GranuloConfig(...))` ; c'est `ProjectController.apply_scene` qui les résout ensuite.

4. lmgc90_engine — le pont pylmgc90

4.1 EngineSession

```
session = EngineSession(project)
session.available() # pylmgc90 importable ?
session.materialize(force=False) # → MaterializedScene (lève PylmgcNotAvailable)
session.write_datbox("DATBOX")
session.emit_pre_script("pre.py") # toujours disponible (délègue au core)
session.emit_chipy_script("command.py", dt=1e-3, nb_steps=5000)
session.export_all(dir, try_datbox=True)
session.visu_avatars() # pre.visuAvatars sur les corps dessinables
session.run_granulo(config) # dépôt pylmgc → ParticlePopulation
session.mark_dirty(); session.dirty; session.body_map; session.population_map
```

Un drapeau `dirty` permet de différer `materialize()` (thread de travail). Le contrôleur appelle `mark_dirty()` après chaque mutation.

4.2 materialize.py

`materialize_project(project)` → `MaterializedScene` construit dans l'ordre : matériaux → modèles → avatars AoS → populations → lois → see-tables → opérations DOF → post-traitement. `MaterializedScene` contient les six conteneurs `pre.*` et les cartes d'identité :

```
body_by_avatar_id: dict[str, body]
bodies_by_population_id: dict[str, list[body]]
material_by_name, model_by_name, law_by_name
```

`_make_material` normalise les options vers l'API pylmgc90 (alias `alpha`→`dilatation`, `sigc`→`iso_hard`, orthotropie en listes, `DISCRETE` en vecteurs, etc. ; voir `lmgc90_engine/tests/test_material_properties.py`). `_apply_dof` n'accepte que les kwargs connus et retire les métadonnées Studio (`phase`, ...).

4.3 avatar_factory.py

`build_avatar(av, materials=, models=)` : un `if` par `AvatarType` (disque, sphère, jonc, polygone, polyèdre, cluster, cylindre, plan, murs 2D/3D, maillé, avatar vide/brique). `build_population_bodies(pop, ...)` : une boucle `pre.rigidxxx` par particule (limite structurelle de pylmgc90). `_apply_contactors` attache les contacteurs (traduction POLYG/POLYR régulier/plein, `shift`, groupes de maillage). `resolve_mesh_filepath` convertit `.brep/.step/.iges/.geo` en `.msh` via `gmsh` (optionnel).

4.4 granulo.py, datbox.py

- `deposit_population(config)` appelle `pre.depositInBox2D/3D, ...Drum2D/Disk2D, ...Couette2D, ...Sphere3D, ...Cylinder3D` et retourne une `ParticlePopulation`.
- `datbox.write_datbox(scene, path)` applique `patch_numpy_cross()` puis `pre.writeDatbox(...)`. `write_cell_dual_datbox` écrit `DATBOX_SPRD/ + DATBOX_STBL/ + PHASES.json` (scènes « cellule »). `export_pre_script` ne nécessite pas pylmgc90.

4.5 Erreurs

`EngineError` ← `PylmgcNotAvailable`, `MaterializationError`, `UnknownBodyError`, `DatboxError`.

5. lmgc90_gui — le client PyQt6

5.1 ProjectController

Façade mince (`QObject` si Qt présent, sinon objet simple grâce à `signals.py`) :

```
ctrl = ProjectController() # fonctionne sans Qt (tests headless)
ctrl.project # lmgc90_core.Project
ctrl.session # EngineSession
ctrl.prefs, ctrl.journal # Preferences, AppJournal
ctrl.state_changed / project_loaded / error_occurred # Signal
with ctrl.batch(): ... # un seul state_changed à la sortie
```

Familles de méthodes :

Famille	Méthodes
Projet	<code>new_project</code> , <code>load_project</code> , <code>save_project</code> , <code>apply_scene(builder, on_conflict=)</code>
Ajout générique	<code>add(obj) + add_material/model/avatar/law/visibility/dof/postpro/population</code>
Édition	<code>update_material/model/law/avatar/visibility/dof</code> , <code>remove_*</code>
Génération	<code>deposit</code> , <code>run_granulo_pylmgc</code> , <code>apply_loop</code> , <code>apply_for_loop</code> , <code>apply_masonry</code>
Groupes / variables	<code>group</code> , <code>remove_group</code> , <code>set_dynamic_var</code> , <code>remove_dynamic_var</code>
Historique	<code>undo</code> , <code>redo</code>
Export / moteur	<code>emit_pre_script</code> , <code>emit_chipy_script</code> , <code>export_all</code> , <code>prepare_run_directory</code> , <code>materialize</code> , <code>write_datbox</code> , <code>visu_avatars</code> , <code>pylmgc_available</code>
Préférences	<code>save_preferences</code> , <code>set_work_dir</code> , <code>work_dir</code>

Toute erreur de validation est journalisée, émise via `error_occurred` puis relancée.

5.2 Vues

- `main_window.create_main_window(controller)` : menus (Fichier, Projet, Édition, Outils, Calcul, Onglets, Exemples, Aide), barre d'outils, dock *Model tree* (gauche), dock *History* (droite), registre `all_tabs` (id → titre, widget, icône).
- **Onglets par défaut** : `material`, `model`, `avatar`, `contact`, `visibility`, `dof`, `viewer.material` et `model` sont **essentiels** (non fermables). `Ctrl+1..9` ouvrent les neuf premiers de la liste Onglets → Ouvrir.
- `tree_view` : lecture seule ; il ne connaît pas encore de nœud dédié aux populations autre que « Populations (SoA) ».
- `history_dock` : liste `project.history.describe_stack()` + boutons Undo/Redo.
- `viewer_3d.Viewer3D` : rendu paramétrique PyVista ; modes Nav / Sélection / Règle ; couleurs LMGC90 / type / matériau / origine ; DOF et lois (see-tables) en surimpression ; export PNG. Le rafraîchissement est **manuel** (bouton *Rafraîchir la scène*).

5.3 Onglets : pattern « factory »

Chaque onglet est une **fonction** `create_xxx_tab(parent=None)` qui importe PyQt6 localement et retourne une classe `QWidget + BaseTab`. Ainsi `import lmgc90_gui` ne requiert pas Qt.

```
def create_xxx_tab(parent=None):
    from PyQt6.QtWidgets import QWidget, ...
    class XxxTab(QWidget, BaseTab):
    def on_state_changed(self): ... # relit controller.project
```

```
return XxxTab(parent)
```

BaseTab fournit `bind_controller` (connexion à `state_changed`), `eval_float/eval_int/eval_list/eval_dict` (virgule décimale FR acceptée, expressions via `safe_eval`), `add_expression_help_label`. Les onglets de liste utilisent `widgets/entity_tree.create_entity_tree`. Les noms par défaut (≤ 5 caractères) viennent de `utils/naming.py` (`unique_name`, `suggest_*_name`).

5.4 Dialogues et assistants

Même pattern `create_*_dialog(...)` : `mesh_wizard` (corps déformables), `masonry_wizard`, `dynamic_vars_dialog`, `pipeline_dialog` (SLURM), `preferences_dialog`, `chippy_routines_dialog` (7 onglets : Modèle, Routines, Extraction, Pilotage, Inspect. 2D/3D/Interact.), `compute_log_dialog`, `journal_dialog`, `examples_dialog`, `link_visibility_dialog`, `command_palette_dialog` (Ctrl+K), `about_dialog`.

5.5 Workers

`compute_worker` (sous-processus `python command.py`, signaux `line_out/finished/failed`), `datbox_worker` (`pre.py` + `command.py` + DATBOX hors thread UI), `materialize_worker`. Pattern `Runner = QThread + QObject` déplacé dans le thread.

5.6 Utils

Module	Rôle
<code>safe_eval.py</code>	évaluateur AST restreint (opérateurs, comparaisons, <code>if</code> ternaire, compréhensions à une clause <code>for</code> , appels sur noms du contexte, attributs non privés)
<code>eval_context.py</code>	<code>build_eval_context(project) : avatar[i], group[...], material[...], model[...], avatars_by_*</code> , math/numpy + variables dynamiques résolues dans l'ordre
<code>preferences.py</code>	Preferences (JSON ~/.lmgc90_gui/preferences.json)
<code>app_journal.py</code>	journal circulaire + fichier optionnel
<code>field_forms.py</code>	schémas de paramètres d'avatars/DOF, <code>parse_vertices</code>
<code>scene_geometry.py</code>	géométrie 2D schématique (sous-échantillonne les grosses populations)
<code>compute_script_generator.py</code>	shim déprécié → <code>lmgc90_core.compute_script</code>

5.7 Exemples

`lmgc90_gui/examples/__init__.py` ne contient que des **métadonnées** (`ExampleSpec(id, title, category, description, dimension, difficulty, scene, tags)`) ; la géométrie est dans `lmgc90_core.scenes`. Une assertion au chargement vérifie `SCENE_BUILDERS[id] is spec.scene`. Le chargement passe par `MainWindow._load_example` :

- **Remplacer** : `new_project(id, dim)` puis `apply_scene(scene)`.
- **Ajouter** : même dimension requise ; `apply_scene(scene, on_conflict="merge")`, historique conservé.

6. Flux de données

6.1 Création d'un avatar

```
AvatarTab._on_add()
→ pre.rigidDisk(...) # entité pure
→ controller.add_avatar(av) → controller.add(av)
→ Project.add(av) → _check_avatar (modèle/matériau existant, validate_avatar)
→ _run(AddAvatar) → CommandHistory.do → Project._insert_avatar
→ session.mark_dirty()
→ state_changed.emit()
→ tous les onglets / tree / history se relisent depuis controller.project
```

6.2 Dépôt granulométrique

```
GranuloTab._on_deposit()
→ GranuloConfig(...)
■ « distribution seule » → controller.deposit → Project.deposit (rayons dans config.radii)
■ case « pylmgc » → controller.run_granulo_pylmgc → session.run_granulo
■ → engine.deposit_population → project.add(population)
■ défaut → controller.deposit
■ pylmgc dispo → session.run_granulo (repli numpy si échec)
■ sinon → Project.deposit → NumpyGranulo
```

6.3 Sauvegarde / chargement

```
save_project(path)
→ io.save_project: project_to_dict → JSON (.lmgc90)
→ save_populations_sidecar(project.populations) → <nom>.populations.npz
→ data["particle_populations_sidecar"] = "<nom>.populations.npz"

load_project(path)
→ io.load_project: valide le schéma, migre v1→v2
→ fusionne meta JSON + arrays .npz → ParticlePopulation
→ reconstruit les listes d'entités (Project(record=False)), restaure les piles d'historique
```

Contrairement à l'ancienne version, il n'y a **pas** de « reconstruction des objets pylmgc90 au chargement » : la matérialisation est paresseuse (`session.materialize()` au moment du DATBOX / de `visuAvatars`).

6.4 Calcul

```
ComputeTab(dt, solveur, E/S) + ChipyRoutinesDialog (routines, extraction, pilotage)
→ get_parameters() → controller.prepare_run_directory / emit_chipy_script
→ pre.py + command.py (+ DATBOX si pylmgc dispo et auto_write_datbox)
→ create_compute_worker → subprocess « python command.py » (cwd = répertoire)
→ lignes → journal de calcul + journal applicatif
```

7. Systèmes clés

7.1 Identité

Un `Avatar` garde le même `avatar_id` toute sa vie ; groupes, DOF (`target_value`), `Loop.model_avatar_id`, post-traitement s'y réfèrent. Une particule SoA a un id **dérivé** "`{population_id}:{i}`" (jamais stocké).

Exception documentée : en `safe_eval`, `avatar[i]` est positionnel (commodité d'écriture dans les formulaires).

7.2 Variables dynamiques

`project.dynamic_vars` : nom → expression (ou nombre). Évaluées dans l'ordre d'insertion par `resolve_dynamic_vars` ; une expression non évaluable reste une chaîne. Persistées dans le `.lmgc90`. Les populations SoA **ne sont pas** exposées dans `avatar[...] / group[...]`.

7.3 Historique (undo/redo)

`Project.history` sauvegarde ses piles dans le `.lmgc90` (clé `history`) et un `journal` lisible. Le dock `History` affiche `describe()` de chaque commande.

7.4 Préférences

`Preferences` (dataclass) : paramètres chipy (`dt`, `nb_steps`, `theta`, `tol`, ...), environnement (`python_executable`, `work_directory`, `projects_directory`, `auto_write_datbox`, `confirm_run`), autosave, récents, performance (`show_avatars_in_tree`, `max_tree_avatars`, ...), viewer. `chipy_params()` alimente `emit_chipy_script`.

7.5 Conflits de noms (`on_conflict`)

`error` (défaut) · `skip` (réutilise matériau/modèle/loi existants, ignore see-tables/post-pro dupliqués) · `merge` (idem + union des groupes). Utilisé par « Ajouter au projet » des exemples.

8. Cycle de vie d'un projet

```
1. File → New controller.new_project(name, dimension) → Project + EngineSession neufs
2. Configuration onglets → controller.add_* (undoable)
3. Sauvegarde Ctrl+S → .lmgc90 (+ .populations.npz)
4. Scripts / DATBOX Computation → Generate DATBOX / scripts (Ctrl+F5)
   Tools → Generate pre.py / command.py / Export all
5. Calcul F5 → onglet Compute → sous-processus chipy
6. Rechargement File → Open → io.load_project
```

9. Conventions et patterns

Élément	Convention	Exemple
Classes	PascalCase	<code>ProjectController</code> , <code>ParticlePopulation</code>
Factories Qt	<code>create_*_tab/dialog/wizard</code>	<code>create_material_tab()</code>
Slots Qt	préfixe <code>_on_</code>	<code>_on_add</code>
Entités core	sans état Qt/pylmgc	<code>Material</code> , <code>Avatar</code>
Noms LMG90	≤ 5 caractères (matériau, modèle, loi, couleur, shape)	<code>STEEL</code> , <code>rigid</code> , <code>BLUEX</code>
Imports lourds	locaux (dans les fonctions)	<code>from PyQt6...</code> dans <code>create_*</code>

Gestion d'erreur côté vue : `try/except (ValidationError, ValueError) → QMessageBox.warning`. Le contrôleur relance après avoir émis `error_occurred`.

10. Guide de contribution

Ajouter un type d'avatar

1. `types.py` : valeur d'`AvatarType`, et ajout à `RIGID_AVATARS_2D/3D` si rigide ; contacteurs compatibles si besoin.
2. `pre.py` : fabrique `rigidXxx(...)`.
3. `validate.py` : cas dans `_check_geometry`.
4. `pre_script.py` : cas dans `_emit_avatar` (sinon `av = None + TODO`).
5. `lmgc90_engine/avatar_factory.py` : cas dans `build_avatar`.
6. GUI : `utils/field_forms.AVATAR_PARAM_SCHEMA`, `AvatarTab._build`, `viewer_3d._MESH_BUILDERS`, `scene_geometry.avatar_to_geoms`, `visibility_tab._AVATAR_CONTACTORS`.
7. (SoA) `types.POPULATION_TYPES`, `population._validate_extra`, `avatar_factory.build_population_bodies` (`fn_map`), `pre_script._emit_population`.
8. Tests : core (validation), engine (stub de `pre`), GUI headless.

Ajouter une loi de contact

`types.ContactLawType` + catégorie dans `CONTACT_LAW_CATEGORIES` + `CONTACT_LAW_PROPERTY_SCHEMA` (la `ContactTab` construit son formulaire à partir de ce schéma). L'engine transmet `properties` en kwargs à `pre.tact_behav(_make_law)`.

Ajouter un type de matériau / une option

`types.MATERIAL_PROPERTY_SCHEMA` (+ `MATERIAL_PROPERTY_CHOICES`), puis `engine/materialize._MATERIAL_OPTIONS` et `_MATERIAL_PROP_ALIASES`. Ajouter un test dans `test_material_properties.py`.

Ajouter un onglet

1. `views/tabs/mon_tab.py` avec `create_mon_tab()` (pattern §5.3).
2. `main_window.py` : import, instantiation, entrée dans `all_tabs`, liste du menu *Onglets* → *Ouvrir*, tuple `_all_tabs` (pour `bind_controller`).
3. `views/styles.py` : icône dans `TAB_ICONS`.

Ajouter un dialogue / assistant

Fonction `create_xxx_dialog(controller, parent=None)` dans `dialogs/`, appelée paresseusement depuis le menu ; passer par `controller.batch()` pour les créations multiples, et `controller.journal` pour tracer.

Ajouter une préférence

Champ dans `Preferences` (valeur par défaut), widget dans `preferences_dialog` + ligne dans `apply_to`. `from_dict` ignore les clés inconnues (compatibilité ascendante).

Ajouter un exemple

1. Fonction `ma_scene(project: Project) -> None` dans `lmgc90_core/scenes/demos.py` (entités uniquement, helpers de `_common.py`) ; l'enregistrer dans `SCENE_BUILDERS` et `scenes/__init__.py`.
2. Entrée `ExampleSpec(...)` dans `lmgc90_gui/examples/__init__.py` (import de la scène).
3. Documenter dans `examples.md`.

Ajouter un générateur de masse (SoA)

Produire `centers (N, dim)` et `radii (N,)`, créer la population via `ParticlePopulation.create(...)`, puis `project.add(pop)` (ou `_insert_population`). Vérifier `POPULATION_TYPES`, l'émission `pre.py` et la matérialisation.

11. Tests

```
cd lmgc90_core && python -m pytest -q # headless : pureté, projet, population, io, validation
cd lmgc90_engine && python -m pytest -q # sans pylmgc90 (stubs) ; marqueur « pylmgc » sinon
cd lmgc90_gui && python -m pytest -q # contrôleur headless ; certains tests Qt en
QT_QPA_PLATFORM=offscreen
```

Tests structurants : `test_purity.py` (core), `test_engine_purity.py`, `test_wave1_scene.py` (parcours complet scène → export → save/load), `test_wave5_assistants.py` (maçonnerie, déformable), `test_visibility_suggestions.py`, `test_avatar_tab_generation.py` (Qt offscreen, `importorskip`).

12. Points de vigilance connus

Relevés à la lecture du code actuel (à confirmer / corriger) :

- **Persistance partielle** : `io.project_to_dict` n'enregistre que les avatars `origin == manual`. Les avatars issus de `apply_loop`, `apply_for_loop` ou de la maçonnerie (`expand_masonry` pose `origin=FACTORY`) ne sont donc pas relus à l'ouverture, alors que les groupes qui les référencent le sont. Les `for_loops` ne sont pas écrits dans le JSON.
- **Undo non uniforme** : `apply_for_loop`, `apply_masonry`, `deposit` (via `_insert_population`), ainsi que `remove_material/model/law/visibility/dof/postpro/population` et `update_*` du contrôleur modifient le `Project` sans `Command`. Le test `test_deposit_is_soa_and_one_undo` suppose pourtant un undo unique du dépôt.
- **Groupes et DOF dans `pre.py`** : `_emit_dof` pour une cible `group` n'applique l'opération qu'au premier membre (commentaire « apply to each member »). Le moteur (`_apply_dof`) boucle bien sur tous les membres.
- **Groupes de populations** : `EngineSession.run_granulo` ajoute aussi le `population_id` dans `avatar_groups` (doublet avec `population_groups`). `compute_script._resolve_group_ids` ne résout que les avatars AoS (`index + 1`) et ne gère pas les populations.
- **Émission des populations** : `pre_script._emit_population` ne connaît que disque, sphère, disque discret, cluster, cylindre ; polygone/polyèdre retombent sur `rigidDisk`.
- **Viewer** : `_render_via_pylmgc90` importe encore `...core.pylmgc_bridge` (ancienne architecture) ; le mode « ■ » doit être porté sur `EngineSession`.
- **Chargement de populations** : un sidecar absent ou incomplet ignore silencieusement les populations concernées (plus de `load_warnings`).
- **Non portés** : Particle Factory, convertisseur de script, assistants Projet/Granulométrie, onglet Templates/Bibliothèque.

Annexe — Fichier à modifier selon le besoin

Besoin	Fichier
Nouveau type de donnée / enum	<code>lmgc90_core/types.py</code> , <code>entities.py</code>
Règle de validation	<code>lmgc90_core/validate.py</code>
Nouvelle action annulable	<code>lmgc90_core/commands.py</code> + <code>Project</code>
Format de sauvegarde	<code>lmgc90_core/io.py</code>
Contenu de <code>pre.py</code>	<code>lmgc90_core/pre_script.py</code>
Contenu de <code>command.py</code>	<code>lmgc90_core/compute_script.py</code>

Besoin	Fichier
Appel pylmgc90	lmgc90_engine/avatar_factory.py, materialize.py
DATBOX	lmgc90_engine/datbox.py
Logique GUI partagée	lmgc90_gui/controller/project_controller.py
Un onglet	lmgc90_gui/views/tabs/<nom>_tab.py
Vue 3D	lmgc90_gui/views/viewer_3d.py
Évaluation d'expressions	lmgc90_gui/utils/safe_eval.py, eval_context.py